

EECS Student Magazine

Works from UC Students at the University of Cincinnati



CONTENTS:

- FOREWORDS FROM DR. CAHAY AND OMAR ALSAYED
- TEN ARTICLES AND TWO ABSTRACTS

Contents

1	Forewords	2
1.1	Foreword from Marc Cahay	2
1.2	Foreword from Omar Alsayed	3
2	Articles	4
2.1	Technical	4
2.1.1	A Comparison of Sequential vs GPU Optimized Breadth First Search using Astronomy Data	4
2.1.2	Electrochemical Aptamer-Based Sensor Development at Novel Device Lab	7
2.1.3	Reliability Approaches for Integrated Circuits	10
2.1.4	Riptide: A Dynamic Neural Network Framework	13
2.1.5	Software Testing Techniques and Methodologies	16
2.1.6	The Identity of Programming Languages	19
2.1.7	Two Modern Solutions to the Minimal-Time Control of Inverted Pendulum Problem .	22
2.2	Experience	26
2.2.1	Design of a Learning Module for Remote Areas	26
2.2.2	Having the Entrepreneurial Mindset as a Computer Scientist	28
2.2.3	Unity Game Development as a Computer Science Student	30
3	Abstracts	32
3.1	Abstracts from Ahmed Youssef and Siddharth Barve	32

Foreword from Marc Cahay

EECS Department Head

Thanks to the hard work of Omar Alsayed and Chloe Muncy, the Department of Electrical Engineering and Computer Science (EECS) is able to release its 3rd Edition of the EECS Student Magazine. There was a delay in releasing this edition partly because of the COVID pandemic. This edition contains ten articles in which students from different backgrounds relate some of their research and other experiences while attending the University of Cincinnati. It is nice to see that more students are involved in various research projects with many professors in the College of Engineering and Applied Science.

Research in all engineering fields is progressing very rapidly. In the future, we need to make sure we provide more research experience for our undergraduate students as early as possible in their education. I would like to thank all the contributors and the faculty advisors who supervised the students and helped them carry out their research projects. Great job!

Foreword from The Editorial Board

Omar Alsayed, *MS, Comp. Sci., BS, Comp. Eng.*

Exploration is ultimately an endeavor into the unknown, with the goal of analyzing all that we are enclosed by. To achieve this, diligence, tenacity, and some luck are needed. What is truly astonishing about what we do, whether from a research or engineering practicum perspective, is that anyone who works hard to innovate has the chance to augment the boundaries of what we now know as science and engineering – with the support of great professors and fellow students. Our highly qualified faculty members are experts in many topics ranging from quantum computing to deep learning to robotics and more.

Since 2018, our IEEE group has been contributing to a community of doers by allowing them to showcase their work, whether it is a conference experience, a senior design project, or whatever accomplishment it may be. Our community is led by professors, researchers, and student leaders that are always willing to help. But it always starts with *you*. I urge all my peers to reach out, become more involved in the community, and ultimately become part of this publication in future years.

We have expanded from the niche crowd that we had when the EECS Student Magazine was launched in 2018, and I hope that the future Editorial Board continues on expanding this excellent platform to persist showcasing the great works of students.

A Comparison of Sequential vs GPU Optimized Breadth First Search using Astronomy Data

Tyler Hilbert, *MEng, Comp. Eng.*, Tylor Beatty, *BS, Comp. Eng.*, Matt Nemeth, *BS, Comp. Eng.*

Abstract—This paper compares the run-time of a breadth first search algorithm running on a CPU to a breadth first search algorithm that is optimized to run on a GPU using CUDA. The dataset used consists of a list of astronomy measurements taken from Earth. A graph of galaxies connected by a distance below a threshold was created from this dataset. Then, the two algorithms were tested for run-time efficiency using the constructed graph.

I. AN EXPLANATION OF THE PROBLEM AND DATASET

INTERGALACTIC space travel is technology of the future. Inevitably, we will find a way to travel between galaxies. One of the difficulties that comes with this new ability is path planning. When intergalactic space travel becomes feasible, we will have a maximum distance that can be traveled. For example, we may design spaceships that can travel 1 light year before needing to refuel. But what if you want to travel to a galaxy that is 5 light years away? You will have to make stops at other galaxies along the way.

Our team set out to write a program¹ that can compute the optimal path when travelling across the universe. Our program attempts to find the path with the fewest stops between 2 galaxies. The purpose of this is to stop the least amount of times to refuel. You can think of it as Google Maps for space travel, getting you to your destination galaxy the fastest.

The first challenge is determining the distance between each galaxy. To do this, we relied only on observations that can be made from Earth. Two datasets were stitched together to create a dataset with the following fields:

- 1) **Right Ascension (RA)** – Used to calculate the angle from Earth to the galaxy (ascension).
- 2) **Declination Value (DEC)** – Used to calculate the angle from Earth to the galaxy (declination).
- 3) **Light Years** – Light years from Earth to the galaxy.

Using these 3 values, the distance between 2 observed galaxies can be measured by using the following equations.

Right Ascension:

$$\text{degrees}(\Theta) = 15 \left(h + \frac{m}{60} + \frac{s}{3600} \right) \quad (1)$$

where h is the hour measurement, m is the minute measurement, and s is the second measurement.

Declination Value:

$$\text{degrees}(\Phi) = \text{deg} + \frac{m}{60} + \frac{s}{3600} \quad (2)$$

Galactic Position on the Cartesian Plane:

$$x = r \cos(\Theta) \cos(\Phi) \quad (3)$$

$$y = r \sin(\Theta) \cos(\Phi) \quad (4)$$

$$z = r \sin(\Phi) \quad (5)$$

where r is the distance in light years from Earth.

Distance between 2 Galaxies Given the Coordinates:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \quad (6)$$

Using the equations above, we calculated the distances between galaxies. A graph was created with edges between the galaxies that were within the travelling distance of the spaceship. For example, we assumed that our spaceship can travel 1 light year before needing to stop at a galaxy for a refuel break. Using that assumption, the following graph shown in Figure 1 was made. The edges show the galaxies that are less than 1 light year apart.

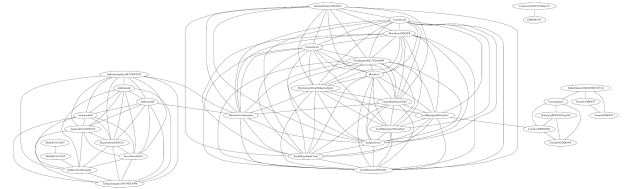


Fig. 1. Visual Representation of the Testing Graph.

It was decided that the overall distance travelled was less costly than the number of stops. The determination of this metric meant that the goal would be to minimize the number of stops. After careful discussion, it was found that a breadth first search would be an ideal algorithm for this problem. As some background, a breadth first search is a searching algorithm that looks outwards 1 node at a time from a source. As a result of this fact, it will always find a solution that has the minimal amount of in-between nodes in a unweighted graph. For a sequential BFS, the time complexity is $O(|V| + |E|)$ which we can consider as $O(n)$ where V is the vertices and E is the edges. Using a parallel breadth first search algorithm, our work complexity value is $O(n^2)$ and our step complexity value is $O(n)$. Using a parallel algorithmic approach, we are able to improve the speed of completion because we are able to use individual threads to check certain paths instead of returning to a point and checking another path on the same thread.

¹Code: <https://github.com/Tyler-Hilbert/SpaceTripPlanner>

II. HIGH LEVEL DESIGN AND EXAMINATION OF THE PARALLELIZED SECTIONS

This project contains three parts. The first is gathering the data from [1] and [2]. The next is a Python3 script used for preprocessing the “GalaxiesWithDistances.csv” file. This is where all calculations are done to create a graph with all the galaxies that are within the specified distance. The preprocessing script then formats the graph data and generates both a CUDA and C++ file that contains graph initialization data. These generated data structures then need to be copied and pasted into the correct CUDA or C++ BFS file. This was done because it was determined that it would be quicker to develop with the assumption that the preprocessor could run entirely separate from the CUDA/C++ programs to alleviate the need for any variable input into CUDA/C++. Various technical resources were utilized [3, 4, 5, 6, 7, 8].

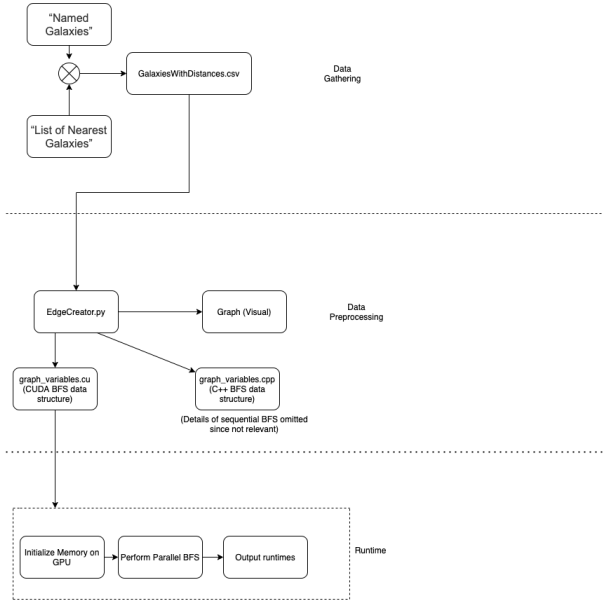


Fig. 2. High Level Flow Diagram.

In Figure 3, an example of the parallelism the algorithm achieves is given. Each step further from the chosen source node is processed in parallel. Depending on the choice of graph, the algorithm can provide large performance improvements. However, for very linear graphs with few branches, it would be expected to run similarly to a sequential algorithm since computations will not be able to be done in parallel if there are not enough edges. A logarithmic improvement was expected assuming that each node has many edges.

Algorithm 1 is the pseudocode for the algorithm’s implementation. The first loop is done on the CPU sequentially and continues until the GPU kernel returns true for a completion flag. The body of this loop is the GPU kernel and is the code that is implemented in parallel for each marked, unvisited node. A technical challenge that was encountered with this code was the potential race condition for the processing of a neighboring/child node, specifically in the context of writing the previous path of nodes. This issue was mitigated using an

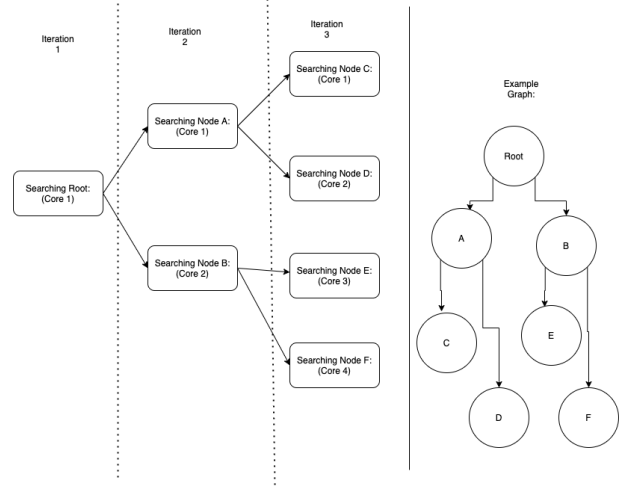


Fig. 3. Example Execution of Parallel BFS Algorithm.

atomic compare-and-swap instruction, ensuring the node could only ever be processed a single time.

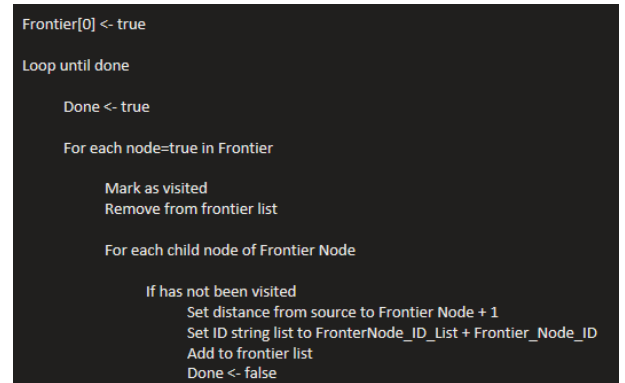


Fig. 4. Text Diagram for Parallel BFS Algorithm.

Figure 4 shows an example of pseudocode for the algorithm’s implementation. The first loop is done on the CPU sequentially and continues until the GPU kernel returns true for a completion flag. The body of this loop is the GPU kernel and is the code that is implemented in parallel for each marked, unvisited node. A technical challenge that was encountered with this code was the potential race condition for the processing of a neighboring/child node, specifically in the context of writing the previous path of nodes. This issue was mitigated using an atomic compare-and-swap instruction, ensuring the node could only ever be processed a single time.

III. RESULTS

To evaluate the parallel performance of the algorithm, a comparison was made between a parallel and a sequential BFS. The run-times of these algorithms were compared using different starting points from the same galaxy dataset. The following equation was used to calculate the performance increases of the parallel algorithm over the sequential algorithm:

$$\text{Performance} = 100 \left(\frac{S - P}{S} \right) \quad (7)$$

TABLE I
TABLE COMPARISON OF PARALLEL AND SEQUENTIAL BFS

	Parallel BFS Average (ms)	Sequential BFS Average (ms)	Performance Increase (%)
00	2.029	8.612	77.000
10	1.572	6.230	74.000
20	1.599	6.750	76.000
25	1.798	7.286	75.000

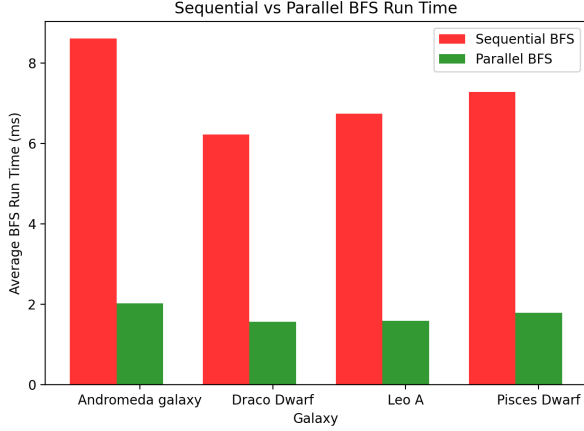


Fig. 5. Graph of Parallel and Sequential BFS Run-times.

where P is parallel run-time and S is sequential run-time.

Figure 5 shows the performance increase of the parallel over the sequential BFS algorithms. For the galaxy dataset used in this report, there was around a 75% performance increase. Please note that the performance increase is dependent on the dataset used. This dataset has 34 nodes and 272 edges. The ratio of edges to nodes is 8:1. One would expect the performance increases to decrease while the ratio gets closer to 1:1 and increase as the ratio approaches ∞ :1. Also note that the performance will start to get diminishing returns as the ratio approaches ∞ :1 due to limitations in the number of cores and other hardware constraints.

IV. CONCLUSION

Overall, the amount of improvement the parallel BFS showed over the sequential BFS was significant and what one would hypothesize to expect. The reason for this performance increase is that a parallel BFS algorithm can compute frontiers being explored that are the same distance away at the same time, while a sequential BFS algorithm has to explore each one separately. This means that the parallel algorithm can examine a larger number of galaxies than the sequential algorithm in the same amount of time. One possible improvement to this paper would involve running the algorithm on a dataset where the spaceship is able to travel different distances. For example, 5 light years instead of 1 light year. This would give a better representation of how the performance of the algorithm will scale. Another possible improvement would be to do the preprocessing on CUDA instead of Python. Theoretically, the edges could be calculated during run-time in parallel instead

of as a preprocessor script that runs sequentially. This was not implemented for this paper because it was assumed that all preprocessing of the galaxy data had already been performed and thus did not need to be optimized.

REFERENCES

- [1] B. F. Madore. *Named Galaxies*. [Online; accessed: 2021-04-06]. 2020. URL: <https://ned.ipac.caltech.edu/level5/CATALOGS/naga.html>.
- [2] Wikipedia Contributors. *List of Nearest Galaxies*. [Online; accessed: 2021-06-20]. URL: https://en.wikipedia.org/wiki/list_of_nearest_galaxies.
- [3] R. Kaszuba. *BFS CUDA*. 2018. URL: <https://github.com/rafalk342/bfs-cuda>.
- [4] P. Harish and P. J. Narayanan. "Accelerating Large Graph Algorithms on the GPU Using CUDA". In: *S. Aluru, M. Parashar, R. Badrinath, V.K. Prasanna High Performance Computing* (2007).
- [5] J. Sanders. "CUDA by Example: an Introduction to General-Purpose GPU Programming". In: *Addison-Wesley/Pearson Education* (2015).
- [6] S. Srinivas. *Implementing Breadth First Search in CUDA*. [Online; accessed: 2021-04-06]. 2017. URL: <https://siddharths2710.wordpress.com/2017/05/16>.
- [7] Udacity. *BFS Try #1 - Intro to Parallel Programming*. [Online; accessed: 2021-04-06]. 2015. URL: <https://youtu.be/SktpTf2MoIw>.
- [8] Udacity. *Merrills Linear-Complexity BFS on GPUs Part 1 - Intro to Parallel Programming*. [Online; accessed: 2021-04-06]. 2015. URL: <https://youtu.be/Dq0ImVxQsRo>.

Electrochemical Aptamer-Based Sensor Development at Novel Device Lab

Ahilya Bali, BS, Biomed. Eng.

Abstract—Novel Device Lab is an electrical engineering research lab at the University of Cincinnati that works on biosensing projects. The primary technology being adapted to biosensors is the electrochemical aptamer-based (E-AB) sensor. The lab is composed of graduate students as well as undergraduate co-op students. This article aims to provide an overview of the lab's current projects from the perspective of a co-op student.

I. WHAT ARE BIOSENSORS?

BIOSENSORS are instruments that measure vitals using inputs from the body. An example would be heart rate monitors in smartwatches. Our lab's overall purpose is to develop a biosensor for continuous monitoring that is adaptable for various target analytes in the body. We specifically want to create a device that can provide consistent signal gain and last at least one week, similar to the industry standard for aptamer sensors: the continuous glucose monitor.

The following analytes, which can be used to gather information on a patient's condition, are the ones that the lab is currently working with:

- Cortisol
 - Cortisol is the hormone released when the body is under stress. This analyte can provide information on when the body is beginning to undergo stress but is not showing any obvious signals.
- Vancomycin:
 - Vancomycin is an antibiotic mainly used to treat *Clostridium difficile* infections in the intestine and staph infections in the colon and small intestine. This drug has a small therapeutic window, and as a result, it is difficult for doctors to determine how much is in the patient's body at one time. A sensitive biosensor would allow health care providers to monitor the level of vancomycin in the patient's bloodstream and adjust the dosage accordingly.
- Phenylalanine
 - Phenylalanine, an essential amino acid for making proteins, is being explored as a treatment option for depression and certain skin disorders. Along with this, phenylalanine is one of the key precursors to the amino acid tyrosine, which is crucial for normal brain function, stress response, and mood regulation. Phenylalanine is an amino acid that is not produced by the body enough, so it is necessary to supplement it with food. By monitoring levels of phenylalanine present in the body, health-care professionals can

better gauge how much they need to supplement the amount already present in the patient.

- Luteinizing hormone (LH)
 - LH is one of the hormones produced by the pituitary gland; it is responsible for controlling the menstrual cycle. Because of this, the exact levels of LH are important for determining the fertility window without relying on qualitative signals from the body (e.g., mood changes, discharge, etc.).

Novel Device Lab is currently working with electrochemical, aptamer-based (E-AB) sensors to develop biosensors for continuous monitoring of the analytes above. An aptamer is a DNA strand (single or double) that is tagged with a redox reporter at one end and chemically bonded to the surface of a gold sensor via a gold-thiol bond on the other end. Sensors are submerged in biofluid—either serum, interstitial fluid, or blood—that contains the target analyte. The aptamer undergoes a conformational change when it interacts with the analyte in the biofluid, causing an increase or decrease in electron transfer between the redox reporter and gold surface. (A conformational change that brings the reporter closer to the gold surfaces increases the possibility of electron transfer.) This electron transfer is measured as current, which is directly correlated to the amount of analyte present in the solution.

An aptamer exists for glucose and it has been well-adapted to sensors, specifically the continuous glucose monitoring device currently on the market. Aptamers for other analytes are not well-adapted; this is what the research at Novel Device Lab is focused on.

II. WHAT ARE APTAMER SENSORS?

A conductive material, usually gold, is deposited onto the surface of a device of choice. The aptamer has a disulfide bond at one end which is reduced to allow a thiol group to form. Next, the aptamer is chemically bound to the surface using a gold-thiol bond—a strong bond. The other end of the aptamer has a redox reporter, like methylene blue. After the aptamer has bonded, a self-assembled monolayer (SAM) is bound to the surface, filling in any large holes left after aptamer deposition. The purpose of the SAM is to prevent fouling of the gold surface by molecules other than the target analyte. If other substances sit on the surface, the aptamer is inhibited from changing conformation. The most commonly used SAM when electrodes are functionalized is the organic molecule 6-mercapto-1-hexanol (MCH). Aptamer sensors can be adapted for various structures (planar, microneedles, rod/bulk).

A voltage is applied to the electrode at a specific frequency to induce the flow of electrons. This is done with a potentiostat (from CH Instruments). For the purpose of this explanation, the target analyte is cortisol and the redox reporter tagged to the aptamer is methylene blue (aptamer is bound to a gold electrode). The electrode is submerged in biofluid where it interacts with cortisol. As cortisol comes close to the surface of the electrode, it interacts with the aptamer. This interaction induces a conformational change, causing methylene blue to come closer to the electrode surface. When this happens, methylene blue is reduced (gains two electrons). This exchange of electrons is measured as current, which is the output signal. A titration curve between the concentration of cortisol and the current measured can be derived through experimentation.

For each voltage, a current response is measured. The potentiostat applies a range of voltages and measures the corresponding current response. For a small window within the range of voltages, there is an increase in current. This is referred to as the “peak.” The resulting graph, called a voltammogram, compares current measured for each voltage applied. The peak signal increases or decreases with changes in the concentration of the target analyte.

Aptamers can be customized for any target analyte. While the process (called SELEX) can be long, the end result is a customized DNA aptamer.

Interactions between the DNA aptamer and an analyte are reversible, unlike enzymatic aptamers which are more common. Enzymatic sensors rely on the chemistry that occurs between the enzyme and the analyte. This interaction is irreversible, making enzymatic sensors a “one-time-use” device. DNA aptamer sensors allow for a reversible interaction, making the sensors more versatile.

III. WHAT WORK DO WE DO AT NOVEL DEVICE LAB?

E-AB sensors have been adapted to various types of electrodes: rod electrodes (a.k.a. “bulk” electrodes), planar electrodes, and microneedles. Rod electrodes are typically used to run tests but are not practical to make a device.

A. Planar Electrodes

One electrode structure that is more practical for devices is the planar electrode. Unlike the rod electrodes, these start as small glass slides and are manufactured into usable electrodes. To adhere the gold to the glass slide, titanium is deposited directly on the slide and the gold is deposited on the titanium. The deposition of metal is done with a technique called sputtering. The metal is deposited in a specific design with large circles on one end to act as contact pads and small circles on the other end where the electrode is functionalized with the aptamer. Planar electrodes are not flexible, but they are being developed as a point-of-care device. A sample of biofluid is taken from the body and tested on an external device. With the success of point-of-care devices, flexible electronics for a wearable biosensor can be developed.

B. Adapting Microprocessors to Planar Electrodes

As mentioned before, a range of voltages is applied to the electrode and the output current is measured for each voltage. For a small window of voltages within this range, a visible increase in current can be observed. This is referred to as the “peak.” Based on the redox reporter tagged to the aptamer, the window of voltages for which the peak is observed changes. It also changes over time as the sensor surface changes with natural degradation.

In addition to natural degradation, repeatedly scanning electrodes at a wide range of frequencies can damage the sensor over time. The goal of using a microprocessor is to program it to scan the electrode with voltages only in said small window. The challenge faced is that the small window where the peak is visible changes over time. This is where the research currently is.

C. Microneedle Sensors

Before discussing microneedle sensors, it is important to understand the layers of the skin. Interstitial fluid (ISF) is found everywhere in the body within the cell-matrix, and it carries some analytes in the body such as cortisol. The epidermis is the topmost layer of the skin and is avascular, although it has some interstitial fluid. The dermis is the vascular middle layer and is the richest in ISF. The hypodermis is below the dermis and has larger blood vessels compared to the dermis. Since the dermis is ISF-rich, the goal is for the needle to penetrate the dermis and access abundant ISF.

Unlike planar electrodes, microneedle sensors are currently being developed for wearable biosensors. There are three designs for microneedle biosensors:

- 1) A hollow microneedle is inserted into the skin to penetrate the dermis. The functionalized gold sensor with aptamer sits above the skin with adhesive. ISF flows from the dermis into the needle and travels up to the aptamer sensor.
- 2) A microneedle is plated in gold and functionalized with aptamer. Like design one, the microneedle penetrates the dermis to access abundant ISF. The aptamer on the needle interacts directly with ISF in the dermis. The needle can be hollow, to increase surface area, or solid.
- 3) A stainless steel acupuncture needle, which is longer and thinner than a microneedle, is sputtered with gold and functionalized with aptamer. The needle penetrates the dermis (deeper than a microneedle), and ISF interacts directly with the aptamer on the needle. This design is in the testing process where we are determining if a signal can be discerned from the output current.

D. Inorganic Blocking Layer

The continuous glucose monitoring technology on the market is functional for up to two weeks. One downfall of using an organic SAM like MCH is that it degrades quickly. Monolayer degradation due to MCH fouling can be observed within a few days of sensor functionalization if conditions are not ideal. Novel Device Lab is working towards characterizing the

conditions under which sensors should be stored to improve longevity to up to a week, as well as exploring other types of blocking layers.

The use of an inorganic blocking layer may be a possibility to increase sensor stability by prolonging the degradation of the monolayer. Currently, our lab is exploring the use of dielectrics as potential blocking layers, specifically silicon dioxide and aluminum oxide. Electron transfer from conformational changes of the aptamer can still occur due to microscopic defects in the dielectric monolayer deposited on the gold.

Biosensors for continuous monitoring of analytes have the potential to change how physicians and patients monitor their health. The ultimate goal is to get a device on the market for the purpose of continuous monitoring in a healthcare or at-home setting. As of now, we are in the preliminary stages of figuring out stability and longevity of aptamer technology on biosensors.

IV. HOW HAVE I CONTRIBUTED TO THE PROJECTS?

During my first co-op rotation with Novel Device Lab (summer 2021), I worked on the point-of-care project. I

functionalized planar electrodes with cortisol or vancomycin aptamer to study longevity, stability, the limit of detection, and consistency in signal gain. I also gained skills in biofluid handling, operating a potentiostat, and analyzing voltammograms. This allowed me to directly contribute to data being collected and analyzed.

This semester, I am working on the inorganic blocking layer project. The current goal of the research in progress is to determine if sensors with a dielectric blocking layer have a signal compared to bare gold electrodes, such as the bulk electrodes mentioned earlier. Once this has been completed, further research can be done on the limits of an inorganic blocking layer, such as limit-of-detection, sensitivity, and longevity. So far, I have learned how to run stability tests using electrochemical impedance spectroscopy (EIS) and run titrations with redox reporters methylene blue, hexaammineruthenium(III) chloride (RuHex), and ferro ferricyanide.

Working at Novel Device Lab has been a great opportunity to learn skills in research and biosensing applications. Seeing class concepts in a research setting and being used to develop technology used in the field was a valuable experience.

Reliability Approaches for Integrated Circuits

Greg Muha, *BS, Elec. Eng.*

Abstract—High-performance integrated circuits can contain upwards of billions of transistors, all of which must maintain sub-nanosecond timing windows in synchronous designs in order to avoid failure. However, there are several unpredictable factors which require circuit designers to reduce performance in favor of continued reliability, such as aging, process variations, workload, and temperature. In-situ monitors, when combined with adaptive compensation techniques, are a solution which have demonstrated the ability to reduce conservative clock speeds and increase the lifetime of integrated circuits. This work will review reliability issues and solutions and describe a simulation method which can be used to predict the effectiveness of a critical-path timing-monitor combined with adaptive voltage scaling.

I. INTRODUCTION

THE continual trend of shrinking transistors and increasing performance in integrated circuits has demanded greater engineering complexity in order to meet design requirements [1]. Reliability concerns, such as aging, process variation, and temperature, are becoming more severe with smaller transistors. This results in unpredictable delay and power consumption, which requires designers to take a conservative design approach and trade performance for reliability.

Timing errors are a cause of failure in synchronous circuits, which use memory elements that launch and capture data in sync with a clock signal. Fig. 1 shows a timing path between two flip-flops, which are a 1-bit memory element that can change state on an active clock edge.

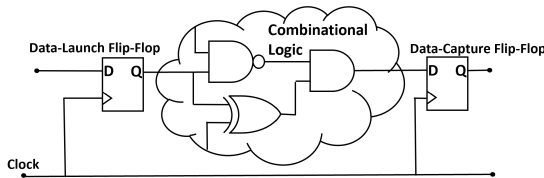


Fig. 1. Data path for a sequential circuit

If data reaches a capturing flip-flop too late, it will not be captured and result in a setup-violation. Conversely, if launched data reaches a capturing flip-flop too early, it will corrupt the data that was just launched by the capturing flip-flop resulting in a hold-violation. Fig. 2 shows the time before and after a clock edge which equate to the setup and hold constraints where data cannot arrive at a capturing flip-flop in order to avoid errors.



Fig. 2. Setup and hold constraints for synchronous logic

On an active clock edge, the data at the D-pin of the data-launch flip-flop is passed to the Q-pin. This delay is called the clock-to-Q delay denoted by $T_{clk \rightarrow Q}$. The data must then propagate through the combinational logic, whose delay is denoted by T_{logic} . The launched data must reach the data-capture flip-flop before the next active clock edge minus the required setup-time (T_{setup}). This is called a setup-timing constraint and is represented by equation 1. The data cannot arrive too early to satisfy the hold-constraint represented by Equation 2.

$$T_{logic} \leq T_{clk \rightarrow Q} + T_{period} + T_{setup} \quad (1)$$

$$T_{logic} \geq T_{hold} - T_{clk \rightarrow Q} \quad (2)$$

Equations 1 and 2 give the range in which the combinational logic delay can exist in order to avoid errors. Since aging gradually increases the propagation delay through logic gates, setup-timing failures are a greater concern to long-term reliability. A solution to aging is to initially decrease the propagation delay through logic gates (T_{logic}) by operating at a higher supply voltage. This comes at the cost of higher power consumption and temperature which can also accelerate aging. The clock period can also be increased which results in an undesirable reduction in performance. Alternatively, in-situ monitors can be used to adaptively adjust the performance and timing-constraints by changing the clock speed or supply voltage as the circuit ages. This achieves the best trade-off between performance and reliability [2] but requires greater initial design efforts.

II. RELIABILITY ISSUES

There are several factors that must be accounted for when designing integrated circuits. Aging degrades the performance of transistors. Negative-Bias temperature instability (NBTI) is a significant aging mechanism where PMOS transistors undergo stress degradation that results in an increased threshold voltage when they are turned on (low signal at the gate). They undergo a recovery period when the gate signal goes high, but not to the degree that it was stressed resulting in a gradual increase in threshold voltage. The net change in threshold voltage are modeled in equations from [3]. The threshold voltage degradation also greatly depends on the signal probability, which depends on the input vector into the circuit. Aging is also dependent on temperature, which varies with the workload and environmental surroundings. Additionally, process variation can introduce random variations in transistor parameters. These combined effects can result in great uncertainty in an integrated circuit's operation throughout its lifetime.

III. ADAPTIVE TECHNIQUES

There are a couple techniques that can be implemented to adaptively change the circuit's operating parameters based on inputs from in-situ monitors. Voltage scaling involves increasing the main supply voltage to the transistor. Body Biasing, also known as bulk biasing, is a technique to adjust the threshold of a transistor for either smaller delay or lower power by applying a bias voltage on a transistor body. Both of these methods require additional voltage control circuitry and can be used in conjunction with frequency scaling to change the clock period.

In-situ monitors are the feedback components of an adaptive system. A certain type of monitor is the critical path replica, whose propagation delay is measured as the circuit ages. Another method includes the critical path monitor, which directly measures the increased delay on timing-paths from aging. One such monitor is the Razor circuit [4], which was modified in [3] to function as an aging-failure prediction monitor. This aging monitor shown in Fig. 3 consists of a multiplexer that selects from varying delays to add to the shadow flip-flop. When the added delay causes a setup-timing failure on the shadow flip-flop, the data-capturing and shadow flip-flop will output differing values and cause the XOR gate to go high, indicating a nearing setup-failure on the data-capturing flip-flop.

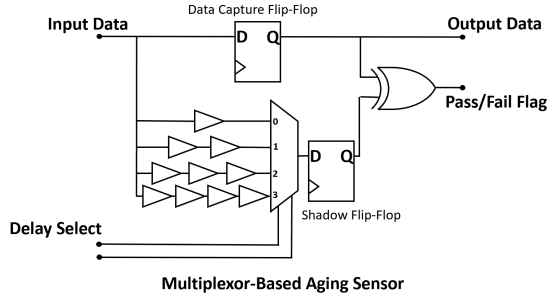


Fig. 3. Razor-based Aging monitor from [3]

A. Simulation

In order to analyze the effectiveness of an adaptive compensation technique with in-situ monitors, it is necessary to simulate the timing, voltage, thermal, and aging degradation of a circuit over its lifetime since they are all dependent on another. The authors of [5] are able to obtain a closed-form model of the aging and voltage profile of a circuit with adaptive voltage scaling. Their formula utilizes the α -power law to calculate the delay of a transistor as a function of the supply and threshold voltage. They use a 1-dimensional RC model for heat dissipation based on the total estimated power consumption. A more accurate but design-specific simulation approach can be used to simulate the transient, voltage, and aging profile based on the standard-cell approach.

The standard-cell approach is the most widely used methodology in VLSI design [6]. For each logic gate and memory element, a netlist based on a transistor model is created

and can be simulated in SPICE (Simulation Program with Integrated Circuit Emphasis). SPICE is a type of software tool that can very accurately simulate circuits by solving the circuit equations Kirchhoff's voltage and current laws [7]. This is infeasible to run for very large circuits with millions of elements, so an alternative approach is used.

In a process called library characterization, SPICE is used to compile look-up tables that give the output slew (transition time) and delay for each logic gate from the corresponding loading capacitance and input slew indices. Additionally, for synchronous memory elements such as flip-flops, timing constraints are characterized, including the setup and hold times. These characterized look-up tables can then be used with static timing analysis (STA). STA is a timing verification method that is independent of the input vector into the circuit and can be run in linear time with the size of the circuit [8]. The output loading capacitance for each cell is determined from the physical layout and library characterization. The STA tool propagates the delay and output transition times of each logic gate by indexing the look-up tables at the capacitance and input slew indices. It checks for all possible setup and hold violations, thereby validating the timing of a circuit, but not its functionality.

Using library characterization and STA flow, the aging equations in [3], and the HotSpot thermal simulation software [9], a transient aging and voltage profile can be obtained for a circuit with adaptive voltage scaling. Libraries can be characterized for V_{dd} , V_{th} , and temperature and interpolation between the look-up tables can be done to obtain the specific operating conditions for each logic gate. The transient voltage and aging degradation can be obtained by iteratively updating the aging degradation and increasing the supply voltage when an aging monitor detects an impending setup-failure. The voltage is increased until the monitor no longer detects a nearing failure and thermal simulation can be done to update the temperature. Fig. 4 shows this iterative process which can be performed for the desired duration.

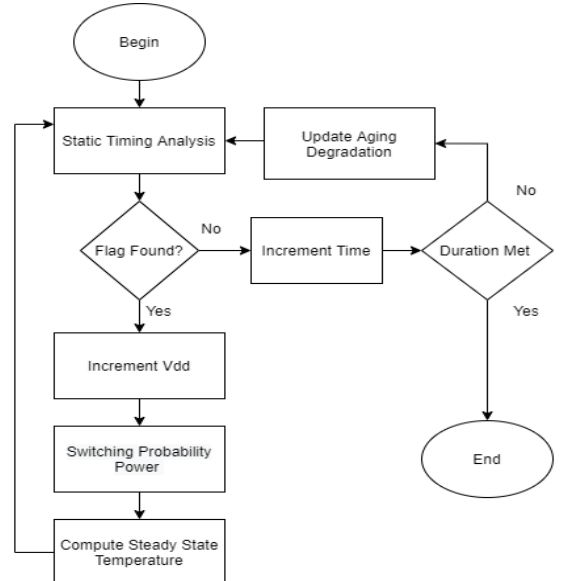


Fig. 4. Adaptive voltage scaling simulation flow

IV. CONCLUSION

Aging and reliability will continue to be primary concerns in integrated circuit design. In-situ monitors can mitigate these design constraints and allow for increased lifetimes at the cost of increased area and design complexity. A simulation method based on the standard VLSI design flow was described which can be used to research the effectiveness of in-situ monitors combined with adaptive compensation techniques.

V. ACKNOWLEDGMENT

This work is supported by the NSF Center for Hardware and Embedded Systems Security and Trust (CHEST) IUCRC under award number CNS 191672.

REFERENCES

- [1] A. Allan et al. “2001 technology roadmap for semiconductors”. In: *Computer* 35.1 (2002), pp. 42–53. DOI: 10.1109/2.976918.
- [2] S. Das et al. “RazorII: In Situ Error Detection and Correction for PVT and SER Tolerance”. In: *IEEE Journal of Solid-State Circuits* 44.1 (2009), pp. 32–48. DOI: 10.1109/JSSC.2008.2007145.
- [3] M. Agarwal et al. “Circuit Failure Prediction and Its Application to Transistor Aging”. In: *25th IEEE VLSI Test Symposium (VTS’07)*. 2007, pp. 277–286. DOI: 10.1109/VTS.2007.22.
- [4] D. Ernst et al. “Razor: a low-power pipeline based on circuit-level timing speculation”. In: *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36*. 2003, pp. 7–18. DOI: 10.1109/MICRO.2003.1253179.
- [5] L. Zhang and R. P. Dick. “Scheduled voltage scaling for increasing lifetime in the presence of NBTI”. In: *2009 Asia and South Pacific Design Automation Conference*. 2009, pp. 492–497. DOI: 10.1109/ASPDAC.2009.4796528.
- [6] A. J. Kessler and A. Ganesan. “An introduction to standard-cell VLSI design: Very large scale integration (VLSI) is becoming an important means of producing electronic circuits at low cost, on tight schedules, and with protection for proprietary designs”. In: *IEEE Potentials* 4.3 (1985), pp. 33–36. DOI: 10.1109/MP.1985.6500265.
- [7] B.J. Sheu et al. “BSIM: Berkeley short-channel IGFET model for MOS transistors”. In: *IEEE Journal of Solid-State Circuits* 22.4 (1987), pp. 558–566. DOI: 10.1109/JSSC.1987.1052773.
- [8] A. Agarwal, D. Blaauw, and V. Zolotov. “Statistical timing analysis for intra-die process variations with spatial correlations”. In: *ICCAD-2003. International Conference on Computer Aided Design (IEEE Cat. No.03CH37486)*. 2003, pp. 900–907. DOI: 10.1109/ICCAD.2003.159781.
- [9] W. Huang et al. “HotSpot: a compact thermal modeling methodology for early-stage VLSI design”. In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 14.5 (2006), pp. 501–513. DOI: 10.1109/TVLSI.2006.876103.

Riptide: A Dynamic Neural Network Framework

Arnav Chandra Komaragiri, *BS, Comp. Sci.*

Abstract—Deep Learning has become one of the most popular fields in Computer Science in recent years, with the expansion of big data and powerful compute engines allowing for intelligent analysis of vast lakes of data. However, the fundamentals behind how Deep Learning as a field functions are often abstracted, leading to existing work operating on a set of fixed frameworks. As such, the author presents Riptide, a Deep Learning framework written from scratch in pure Python and NumPy [1]. Riptide is heavily modeled after PyTorch [2], borrowing the idea of heavy Python integration as well as some syntactical choices. Riptide implements all critical components for deep learning with no extra libraries, demonstrating an understanding of critical components in a neural network. Additionally, the author applies this framework to a reduced version of the MNIST [3] dataset, demonstrating its learning capacity. Furthermore, the design of this new Deep Learning framework allows for more flexible deep learning operations. For instance, this new framework is capable of backpropagating across conditional statements and changing layers, dynamically building computational graphs during the evaluation of the network. Through the design and implementation of this deep learning framework, this work presents a fundamental overview of Deep Learning, as well as how these algorithms are implemented in practice. For reasons regarding the course (Intelligent Systems) this work was done for, no code will be published to maintain academic integrity.

I. INTRODUCTION

DEEP Learning is a field made possible through the availability of vast computational resources and vast quantities of data. With the rise of “Big Data”, organizations have gained the ability to train large neural networks on large datasets. MNIST [3] is one such dataset, being used as a starting hallmark of deep learning in computer vision, and a small benchmark for deep learning tasks. As such, MNIST provides a small benchmark with which rudimentary deep learning systems can be evaluated with minimal computing cost, allowing for the development of a novel deep learning framework named Riptide. The following paper breaks down the functioning of the framework on the MNIST dataset into four parts: a review of neural networks and back-propagation, the Tensor and Module implementation, dynamic computational graphs, and parameter dictionaries/optimizers.

II. BACK-PROPAGATION/NEURAL NETWORK REVIEW

To start, we define a computational graph as a directed acyclic graph (DAG) of tensor nodes, connected by edges representing layers. The source tensor nodes correspond to the inputs of the neural network, while the sink tensor nodes correspond to the outputs of the network. Evaluation of the network can be described as simply traversing across the graph from source to sink node, evaluating layers, and updating downstream node values as necessary. However, the process of “learning” is achieved through back-propagation, an algorithm designed to update parameters in deep neural networks. This

algorithm ultimately reduces down to an abstraction around chain rule, but a review is still useful in the understanding of this system.

To review back-propagation, we define a simple dense neural network architecture involving an input tensor x , hidden neurons h , and output neurons y . Computation of the neural network can be represented as matrix operations, where

$$h = \sigma(W_0x + b_0), \quad y = \sigma(W_1h + b_1),$$

where W_n, b_n are the weight and bias parameters of the n -th layer in the neural network and $\sigma(a)$ is a non-linear element-wise activation function. Evaluation of the neural network becomes straightforward, with the output being

$$y = \sigma(W_1(\sigma(W_0x + b_0))) + b_1,$$

or the composition of the two above intermediary functions. However, back-propagation relies not only on knowing the output of a network, but the gradient of some arbitrary loss l with respect to the parameters W_n, b_n . To do so, we define $l = L(y, \hat{y})$, where L is a differentiable function penalizing network errors from some ground truth y . To start, we can compute the gradient of the loss with respect to the final layer parameters simply, as

$$\frac{\partial l}{\partial W_1} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial W_1}, \quad \frac{\partial l}{\partial b_1} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial b_1},$$

where $z = W_1h + b_1$. To compute the gradients of the loss with respect to the first layer, one can simply evaluate

$$\frac{\partial l}{\partial W_0} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial W_0}, \quad \frac{\partial l}{\partial b_0} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial b_0},$$

where $a = W_0x + b_0$, applying chain rule once again. From here, a common pattern emerges, where the gradient of the n -th layer’s parameters of a network is dependent on the gradient of the loss with respect to the input of the n th layer. We can define this quantity δ_n , where $\delta_n = \frac{\partial l}{\partial u_n}$, where u_n is the input to the n th layer of the neural network. This quantity can be computed incrementally by keeping a running product while traversing backwards across the computational graph. Additionally, one can easily compute the gradient of the loss with respect to a parameter p_{n-1} in the n -th layer of a network by computing $\frac{\partial l}{\partial p_n} = \delta_n \frac{\partial u_n}{\partial p_n}$, thus executing the back-propagation algorithm.

III. DESIGN OF RIPTIDE

A. Tensor/Module Implementation

To start, the atomic type in the Riptide framework is the Tensor, containing information about an N-D array and its appropriate gradient information. Namely, each Tensor

datatype carries a gradient N-D array as well as its value N-D array, allowing for the storage of tensor values and their appropriate gradients in the neural network.

Additionally, the Module datatype is the secondary major datatype in the Riptide framework, corresponding to operations done on Tensors. Some examples of Modules may include dense neural networks, activation functions, or loss functions. Each module provides a `forward()` method, which corresponds to the evaluation of the layer, as well as an optional `get_grad_fn()` method, which returns an appropriate gradient function. This gradient function returned will then return the gradient of the Module's outputs with respect to its inputs for further back-propagation, updating parameters of the Module in place. When the `get_grad_fn` method is not specified, it calls back-propagation on the output tensor, wrapping the entire Module with its appropriate gradient.

Finally, each Tensor datatype has a gradient function, which recursively calls the back-propagation operation on the Module that generated it. When a module is evaluated, it passes its gradient function yielded by `get_grad_fn()` to the output Tensor, allowing for future evaluation of the gradient function and updating of parameters. By organizing information about neural networks in this form, Riptide can flexibly handle both the forward and backward passes of a neural network. Additionally, Riptide allows for custom layers to be easily written by inheriting the Module type, as operation modules can be recursively used inside other Module instances. With this, a flexible dynamic computational graph system can be built for back-propagation, where the forward pass of the network builds the back-propagation computational graph.

B. Dynamic Computational Graphs

As defined earlier, a computational graph is a method of defining computation as a directed acyclic graph connecting tensors and layers. The evaluation or forward pass of the network can be thought of as a breadth-first traversal from source to sink tensor of the graph, whereas the backward pass of the network can be thought of as a breadth-first traversal from sink to source tensor. However, this graph is not required to be constant for the entirety of training; training simply requires that all nodes in the computational graph be accessible by the backward pass at some point during training. Riptide exploits this to build the computational graph of the network during the forward pass, allowing the backward pass to simply focus on the graph built by the last evaluation.

Riptide operates by leveraging function closures to update layer parameters from out-of-scope. Namely, by providing the gradient function yielded by the calling module's `get_grad_fn()` to the output tensor, the output tensor can call its own `backward()` method, which then, in turn, calls the yielded gradient function. This function is then able to access internal variables of the module which returned it despite being called from the Tensor scope, thus leveraging the function closure. Finally, this gradient function returns the running gradient of the final sink tensor with respect to the inputs of the current layer, otherwise known as the error signal for brevity. This gradient function is attached to the output tensor

on the `forward()` call of the module, ensuring that the back-propagation only works on the most recent evaluation of the network. Furthermore, this allows for modules to be evaluated conditionally, with the back-propagation updating the appropriate parameters only as necessary. With this dynamic computational graph, Riptide can provide large flexibility with the design of neural networks, computing necessary gradient information with dynamic network architectures.

C. Parameter Dictionaries and Optimizers

Once gradient information has been computed using back-propagation, this information must be passed to the optimizer to update the parameters of the network. To achieve this, Riptide must first have a method of representing and accessing all parameters in a given network architecture. To achieve this, Riptide acquires a list of all attributes in its class via the `__dict__()` method provided through python, returning a dictionary of all object variables. From there, Riptide then processes that dictionary to get a hierarchical dictionary of pointers to parameter tensors. Sub-modules of a network are represented as nested dictionaries, while single parameter tensors are represented as their entries in the parameter dictionary. From here, this dictionary of pointers is provided to the optimizer class, which then iterates through this parameter dictionary and makes adjustments as necessary based on gradient information. Furthermore, Riptide may also insert temporary tensors for optimizer-based logic alongside each parameter entry in the parameter dictionary, as in the case of optimizers like Stochastic Gradient Descent with Momentum and ADAM (insert adam citation). This representation of parameters also allows for efficient storage of networks, as one can save this parameter dictionary into a file and load it into the model as necessary. With this paradigm, Riptide implements two major optimizers: Stochastic Gradient Descent with Momentum and ADAM [4], along with their DEMON [5] variants with momentum scheduling.

IV. CONCLUSION

This framework was primarily developed during the EECE5136 course at the University of Cincinnati, and was applied to a reduced version of the MNIST dataset to demonstrate effective model training and evaluation. For review, MNIST is a dataset of 28x28 pixel images of handwritten digits [3], where the primary classification task is to identify which digit is written. To demonstrate functionality, the author trained a simple two-layer dense neural network with 200 hidden neurons, intermediate ELU [6] activations, and sigmoid output activations. Mean squared error loss was utilized to train against a one-hot encoded label corresponding to the written digit, thus constructing the image classification problem. ADAM [4] was used to train this model, with a learning rate of 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.99$.

When trained on this reduced MNIST dataset, the network can train to its final test accuracy of 92.4%, thus demonstrating that this framework and this architecture can successfully train simple neural networks. Furthermore, this framework is relatively efficient, being able to train this model for 200

epochs on a 5000 element dataset in approximately 1-2 hours. With this, the author demonstrates the fundamentals behind Riptide, a rudimentary deep learning framework supporting flexible back-propagation and model logic. Furthermore, this paper demonstrates Riptide’s effectiveness on small datasets like MNIST [3], confirming the framework’s capacity to learn.

Future work could involve support for more advanced model functionality like custom learning rate schedules, convolutional layers, recurrent layers, and self-attention layers. Additionally, support for arbitrary loss functions could be added, tying into a more open back-propagation system based on Python operator overloading. Finally, since all core model logic is done in matrix form to leverage fast matrix computation in modern computers, leveraging GPU computation for faster operation is a future step for Riptide. The author hopes this document may be of use to those pursuing a more fundamental understanding of neural networks and deep learning, and for future work to be done by other students engaging in this field.

REFERENCES

- [1] C. R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: <https://doi.org/10.1038/s41586-020-2649-2>.
- [2] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703.
- [3] Y. LeCun and C. Cortes. “MNIST handwritten digit database”. In: (2010). URL: <http://yann.lecun.com/exdb/mnist/>.
- [4] D. P. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980.
- [5] J. Chen et al. *Demon: Improved Neural Network Training with Momentum Decay*. 2021. arXiv: 1910.04952.
- [6] D-A. Clevert, T. Unterthiner, and S. Hochreiter. *Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)*. 2016. arXiv: 1511.07289.

Software Testing Techniques and Methodologies

Ranga Sai Yasaswi Anarajula, *MEng, Comp. Sci., B.Tech, Elec. Eng.*

Abstract—Software Testing is the procedure to discover bugs while executing a program and debugging it so that perfect software is delivered. Software Testing is deemed as the most substantial stage in developing software. It is a way to find and rectify errors, help maintain software and reduce maintenance costs. A key challenge in software testing is the creation of suitable test cases for a system. Even after technical evolution, we still need to make sure software is bug-free before it is launched to market. Software testing is a crucial area of interest and abundant work is put into this field. Whenever developing software, the top priority is given to making the software reliable and bug-free. Testing is generally classified into three levels, Integration Testing, System Testing, and Unit Testing. There exist numerous ways for bug detection. This paper is a detailed study of the most frequently used techniques and tools.

I. INTRODUCTION

SOFTWARE testing deals not only with error detection, but also deals with the behavior of the software under monitored conditions and to confirm that software performs as intended. So, the key functions of testing will be to detect errors, to validate that software performs as the application is theoretically intended. Software testing can be viewed as a process of evaluating the software under various simulated conditions to record an abnormality. It is a method to evaluate the potential of a program and deciding the quality of software. A software quality rubric consists of quality factors like reliability, integrity, security, efficiency, and compatibility. Testing is expensive but little to no testing software can be devastating. Software testing incorporates a protocol that should be met by the software.

II. SOFTWARE DEVELOPMENT LIFE CYCLE

In 2015, [1] stated that SDLC incorporates a set of various guidelines that are followed by the industry of software that aspires to deliver high-quality software [1]. The set of rules in SDLC is not limited to software, they can be extended to hardware too as a perfect system need to have a perfect software along with hardware [1]. SDLC is as follows [2]:

- Planning
- Defining
- Designing
- Building
- Testing
- Deployment

A. Testing

The vital role of testing in SDLC is an evaluation using test cases that are determined by gathering system requirements and user specifications [3] and the sole agenda of testing is to discover bugs and incorporate an algorithm to cross-check

the code such that it delivers its result [4]. A project manager or tester or any software developer or tester can be affiliated in testing. Testing starts with gathering targets required to be met and ends at successfully deploying the system. In the waterfall model, testing will be completed in the phase of testing. Testing the software should stop at a certain threshold as a perfect system is not achievable and the input domain can be huge so testing every domain is not feasible. Expected outcomes to software testing:

1) *Verification and validation*: Verification is a process of authenticating the code without implementing the program like code review or syntax check, whereas validation is checking the product by executing the program.

2) *Priority Coverage*: Certain parts of a system should be made fail-safe and should be given special attention to while testing.

3) *Balanced*: Any testing process should balance out the constraints, technical constraints, and customer anticipations.

4) *Traceable*: A record should be documented to note any outcome, both success and failures of the testing process this prevents redundancy.

5) *Deterministic*: A tester should be aware of the goals, current position, and the expected outcome.

III. SOFTWARE TESTING STRATEGIES

This involves the successful integration of several test cases, design methods into a sequence of actions that result in fruitful testing of any software. Thus, software testing strategies play a vital role in testing. A project manager or a specialized tester or a software engineer or developer can formulate testing strategies. There are four software testing strategies.

A. Unit Testing

Unit testing is accomplished at the bottom tier, which tests the fundamental unit, which can be as trivial as a module. Even the smallest set of lines of code can be considered as a unit and can be tested [5]. In unit testing, the outcome of one module is convoluted with the input of the other if any module fails the while input fails. Repeated unit testing has a cumulative effect and contributes to testing the whole system. It is also named as a white box test class. It complements integration and system-level testing, code reviews and tutorials. Unit testing precedes integration testing and usually carried out by developers. Mocha, Tape, and Jasmine can be some commonly used tools for unit testing [5].

1) Benefits of Unit Testing:

- It is economical.
- It is a reliable source of testing as it reveals latent bugs, provides immense value for a process that is so trivial.
- Includes pipe-lining.

- Achieves parallelism, finding bugs and debugging can take place simultaneously.
- It can detect and debug defects at an early stage at less cost compared to other states which are usually catastrophic.
- Unit testing can be made module-specific or domain-specific [6].

2) *Unit Testing Techniques*: Many useful testing techniques support unit testing. They are divided into three types:

- Functional Testing
- Structural Testing
- Heuristic or Intuitive Testing

B. Integration Testing

Integration testing is a process that involves assembling the program structure while simultaneously handling tests to discover faults correlated with interfacing [4]. Integration Testing is succeeded by Unit Testing but preceded by system testing. The individual modules are merged and are assessed if they are working as intended so the testing can be implemented. Any bug remaining after unit testing is removed after integration testing to maintain a foolproof measure. The key aspect of integration testing is to test the synergy of the system [4]. Integration testing involves a Top-Down approach, Bottom-Up approach. Developers or testers can undertake this procedure. Mocha, Tape, and Jasmine can be few environments to perform this testing.

C. System Testing

System testing is a process that manages to check the overall condition of the entire system. Testing a system depends on the functional specifications of the system [6]. Non-functional quality attributes, such as efficiency, security, and reliability are examined. This testing accumulates all the integrated components required to test and verifies whether it meets the requirements and upholds quality standards. The purpose of this testing is to assess whether the system fulfills specific requirements. Black box testing is implemented in the system. This testing takes place after Integration Testing and before Acceptance Testing. This process should only be handled by independent testers.

Example: When a car is tested, the engine, chassis, gearbox are all tested independently and then a unit testing is performed. Then they are all combined, and integration testing is conducted. When the whole car is assembled, then a system testing is implemented [1].

IV. SOFTWARE TESTING METHODOLOGIES

Accuracy is essential in any software. Accuracy testing needs a rubric, to determine the right behavior from the wrong one. The intricacies of a software module may not be known to the tester while performing a test. Hence, we can implement any of the accuracy testing [1].

A. Black Box Testing

It is a testing strategy that exclusively relies on requests and obligations. Black box testing does not demand information on the intricacies of software being tested. The tester is not required to know the inner functionality of structures in the software. Input and output values alone determine the formulation of test cases, information on design and code is not mandatory. The testers are expected to know the goal but not the way software works. Black box testing can be either functional or non-functional [7]. As the tester does not know the inner side of the system the name black box testing is coined to this procedure. Black Box Testing is alternatively recognized as Functional testing, Data-Driven testing.

Example: Without any knowledge on the inner details of the website, any tester can test the web page, initialize input, execute a test, cross-check the resultant output with the expected result [4].

B. White Box Testing

White box testing demands the understanding of interior paths, code formats, and implementation of the software being tested. This protocol generally requires detailed programming skills as it requires devising test cases based on the code and the tester is familiar with details of the module and can view the intricacies of the module perfectly, hence the name White Box Testing which is also known as Logic Driven, or Path Oriented.

Example: A tester analyzes the code implemented in the system, perform a test case with both valid and invalid inputs and validates the resultant output to the expected outcome and determines if any modifications to the system are necessary. White box testing can be viewed as work of a plumber who needs to know which part needs troubleshooting [2].

C. Gray Box Testing

This strategy demands an understanding of the implementation of the module to be successfully tested. Then the box is shut, and the tester applies any black-box test which is the best fit for the system according to his expertise. This can be viewed as a hybrid of both white and black box techniques; partial knowledge is gained.

D. Black Box Testing vs White Box Testing

The ways a tester can approach a specific type of application differently during testing. While during white box testing, knowledge of the intricacies of the application is a prerequisite to the tester which is not the case for black box testers, they should know the basic accounting of the system [3]. White box testing should be conducted by developers as they are familiar with internal applications. In black-box testing, and overall functionality examination of the application is achieved, whereas, in white box testing, more tedious inspections are performed such as code and architecture reviews, eliminate bad code practices, and carry out component level testing [2]. There are numerous environments accessible in the market for software testing. Some of them are primeval and

some new tools are also developed with a lot of new features. Furthermore, we are going to discuss a tool that is used for automated testing.

V. RANOREX

Ranorex is a tool that is vastly used as it is trivial, compact for automatic testing. It tests applications from a user perspective which makes it a better alternative to other testing tools and employs common programming languages like .NET. As it is coded in pure .NET, there is no necessity to learn the scripting language [4]. Ranorex supports environments Iron Python and is employed by a lot of tech giants and corporate enterprises. Most simulation tools have the same problems which are the dependency of methods on testers to create perfect test cases to ensure complete coverage. The scope of improvement in Ranorex entails, provides flexibility to create individual plug-ins for clients and thereby providing full recognition for their applications.

A. Salient features of Ranorex

- It incorporates image-based recognition
- Has built-in Record-Replay functionality called Ranorex Recorder [2]
- It supports both 32-bit and 64-bit operating systems
- Employs “.NET” framework to be built on
- Flexibility in the test interface
- The recorder provides readability, which can allow scrutiny from other experienced testers when a novice tester performs software check
- Completely flexible and automated
- Compatible with Ranorex Plug-Ins
- UI allows managing test cases and configurations
- Since it uses .NET compiler, which supports the creation of test automation modules
- Test automation can be done in the client’s environment
- It uses standard and modern programming techniques

- Ranorex recorder enables novice testers to create qualified test cases

VI. CONCLUSION

This paper elucidates on the necessity of software testing, Software testing fundamentals, testing methodologies. Software testing is very understated and conducted less rigorously than it should be, struggling to determine the best set of principles for better software testing is the key reason. For a better yield of software, anyone engaged in testing should be acquainted with fundamental software testing limitations and concepts. This paper concludes with fundamentals in black box testing, white box testing, and gray box testing. We also elaborated on a common corporate tool for testing.

REFERENCES

- [1] P. Yadav. *Review Paper on Software Testing*. June 2015. URL: http://ijirt.org/master/publishedpaper/ijirt102003_paper.pdf.
- [2] R. Chauhan. *Latest Research and Development on Software – Inpressco*. June 2014. URL: <http://inpressco.com/wp-content/uploads/2014/07/paper122368-2372.pdf>.
- [3] R. Kumar. *Software Testing Techniques and Strategies*.
- [4] H. Babbar. *Software Testing Techniques*. Mar. 2017. URL: <http://www.coursehero.com/file/50490175/software-testing-techniquespdf>.
- [5] R. S. Pressman. *Software Engineering: A Practitioner’s Approach*. Palgrave macmillan, 2005.
- [6] H. C. Zhang. “Research on New Techniques and Development Trend of Software Testing”. In: *Advanced Materials Research*. Vol. 760. Trans Tech Publ. 2013, pp. 1298–1301.
- [7] A. Bertolino. “Software Testing Research: Achievements, challenges, dreams”. In: *Future of Software Engineering (FOSE 07)*. IEEE. 2007, pp. 85–103.

The Identity of Programming Languages

Jaran Chao, *BS, Comp. Sci.*

Abstract—Programming languages write every piece of software imaginable. However, the theory surrounding them is cloudy and vague. The following project explains in-depth the design choices of creating a programming language. The main purpose of this project is to research what gives a programming language its identity. This research helped influence the design choices of the toy programming language created for this project.

I. INTRODUCTION

THERE are many programming languages, each with its programming paradigms, styles, and features. Many factors contribute to giving a language its identity. Differences in grammar, abstraction, and ideology make each language unique. However, subtle differences in how the language is implemented also affect its identity. This work will tackle these subtle differences and their effects on language identity.

II. COMPILED VERSUS INTERPRETED

The first major distinction between different languages is if it is compiled or interpreted. Each language has its own way of executing source code and almost every language uses a compiler in some form. Some languages such as Python, Ruby, and Lua will compile to an internal bytecode and then run an interpreter on that bytecode. These languages are considered interpreted. Other languages such as C/C++, Rust, and Go will compile ahead-of-time to a native binary that the OS and CPU can understand and execute. And other languages such as Java, C#, and Elixir compile ahead-of-time to a bytecode, which is then interpreted on a language VM. Both of these types of languages are considered compiled however these distinctions can get very blurry and cause confusion.

Let us first define what the terms “compiled” and “interpreted” mean. These definitions are taken directly from *Crafting interpreters* [1] by Nystrom as they directly apply.

- When we say a language implementation “is compiled” or “is a compiler”, we mean it translates source code to some other form but does not execute it. The user has to take the resulting output and run it themselves.
- When we say an implementation “is interpreted” or “is an interpreter”, we mean it takes in source code and executes it immediately. It runs programs “from source”.

For some languages, this is purely black and white. GCC and Clang for C/C++, rustc for Rust, and gc for Go will compile their respective languages to machine code. The executable is then ran directly by the end user, who may not even know which tool or even language was used to compile it. Therefore, these are compilers.

For others such as PHP3 and MRI (Matz’s Ruby Interpreter), the user would run the program directly from the source code. The underlying implementations would parse it

into a syntax tree and immediately execute it by traversing said syntax tree. No other intermediate steps occurred, internally or externally. So, these are most definitely interpreters.

However, more recent languages have landed themselves in more of a grey area by utilizing aspects of both compilers and interpreters. CPython, for example, is a language that is considered to be interpreted from a user’s perspective, however a peek under the hood shows the truth. CPython is both interpreted, but also has a compiler. When running a Python program, the source code is parsed, then compiled into Python’s internal bytecode format, and finally the generated bytecode is then interpreted by the Python Virtual Machine (PVM). Java is another example of this grey area. Java is a language that is considered to be compiled from a user’s perspective. While this is true, the Java compiler, javac, compiles Java source code ahead-of-time into Java bytecode. This bytecode is then stored in .class files which is then interpreted by the Java Virtual Machine (JVM).

In practice, most modern languages utilize both compilers and interpreters in the build process as you can see below:

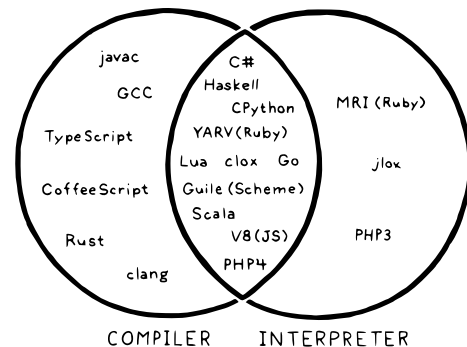


Fig. 1. Venn Diagram of Compiled Languages vs Interpreted Languages [1]

III. COMPILATION TARGET

Now that we understand the differences between compiled and interpreted languages, we can better understand each of their capabilities. Each have their own advantages and disadvantages.

A. Compiling to Native Binaries

Many languages compile to native binaries. C/C++, Rust, and Go are common examples of languages that compile to native binaries. An advantage of native binaries is that they require little to no overhead. Unlike some of the other aforementioned languages, the native binary is able to be executed directly without having to spin up a VM. Because the native binary consists of machine code that is understood by the underlying OS and CPU, no additional software or

translation units are needed. This specificity allows for a compiler such as GCC or rustc to make specific optimizations for specific architectures.

This level of precision, however, is a double-edged sword. While it allows for compilers to make specific optimizations, it also means for a program to run on another platform with a different CPU or OS that program must be recompiled for that specific set platform. This may not be difficult for end users as it is just a couple of CLI commands. For a language creator, this does not scale well as the compiler would need to be rewritten every time the language decides to support a new architecture. Though, this problem can be circumvented by choosing a target language to compile to. Then, instead of writing and maintaining the compiler backend for multiple different architectures, you can utilize the existing compilation tools of the target language as an optimizer and code generator. Most common targets are LLVM IR [2] and C.

In spite of simplifying the compiler for a language, this now also requires that the target platform has the ability to run the target language, introducing a dependency into the language. It is for this reason that the toy programming language did not choose to compile to a native binary as a substantial dependency was not ideal and rewriting the code generation process for multiple architectures was also heavily undesirable.

B. Tree Walk Interpreters

In more recent years, most interpreted programming languages have strayed away from direct traversal of the source code's syntax tree. The main reason is that traversing trees is very slow and very resource intensive. Traversing the syntax tree and executing the code at every node are very costly operations. Storing the tree and all the data required at every node takes a lot of memory, especially for larger programs that can have thousands of lines of active code.

However, the biggest advantage of tree walk interpreters is their simplicity. For student projects and embedded and domain specific languages, this implementation is very common as the speed and memory constraints become less of a concern as the programs will be smaller and simpler. The speed of this implementation was the reason the toy programming language did not choose to utilize a tree walk interpreter.

C. Compiling to IR

Many languages compile to an intermediate representation (IR), usually in the form of language specific bytecode, before running an interpreter. Both interpreted and compiled aforementioned languages alike. Each have their own minor differences, but both types work in almost identical fashion. Every language will first parse and compile the source code into some IR format. This IR is then interpreted by a language VM. However, the key difference is when the IR is interpreted. For languages like Python, Lua and Ruby, the IR is interpreted on their respective language VMs immediately after the compilation step is completed which is why these languages are considered interpreted. However, languages like Java and C# compile to an IR ahead-of-time and then this IR

can be executed by the language VM later which is why these languages are considered compiled.

The biggest advantage of this approach is that optimizations are able to be performed on the IR such as dead code elimination, loop and data optimizations, and tail call optimization just to name a few. These optimizations can greatly increase the speed of the program once it is passed to the interpreter. This approach is also scalable as it allows for the additions of new platforms. The only requirement is to rewrite the language VM which is not as complex as the whole language itself. It was for these reasons that the toy programming language chose to utilize compiling to an IR and having a language VM.

IV. TYPE SYSTEM

The next major distinction is the type system of a language. The type system is the main basis for the rest of the features of a language. Type systems often take one of two forms: static or dynamic. This is a highly contested topic and different sources will cite different definitions for these terms. For this work, these terms will be defined as the following:

- When we say a language is “statically typed”, we mean that the value of the variable or function will be known ahead-of-time. This usually means compile time and therefore most statically typed languages will have an intermediary step which will perform some sort of compilation.
- When we say a language is “dynamically typed”, we mean that the value of the variable or function will be determined by the value stored at runtime.

A. Dynamic Typing

Dynamic typing is most commonly utilized by scripting languages. This speeds up the development process as less time is dedicated to knowing the type of the variables and functions in use, instead allowing for the underlying interpreter to determine if the code is utilizing valid types during execution. However, this approach to typing has many drawbacks. According to “To Type or Not to Type: Quantifying Detectable Bugs in JavaScript”, almost 15% of all software-related errors are type related errors [3]. While this is not conclusive evidence that dynamic typing leads to more error prone code, there is a reason why dynamic languages such as Python [4] and Ruby [5] have started to implement some form of type analysis to ensure type safety.

B. Static Typing

A wide variety of languages utilize static typing for many different applications. Some examples include TypeScript for web applications, C++ and Rust for performance intensive applications, Kotlin and Swift for mobile applications, and many more. As stated before, static typing can allow for less error-prone code. The ahead-of-time type checking can catch many bugs in development. Type checking at compile time also leads to performance boosts. Compared to dynamically typed languages, statically typed languages do not need to do type checking at runtime since it has the guarantee that once

a program is compiled it is a valid program, at least from the type system’s perspective.

This insurance, however, comes with many disadvantages. Conforming to a type system may lead to more verbose programs and increase development time. Telling a compiler the type signature of every variable and function is tedious. However, to combat this, languages such as C++ [6], Java [7], Kotlin [8], and more, implement a form of type inference. This gives software creator a more dynamically typed development experience while not sacrificing the advantages of having a static type system.

V. PROGRAMMING PARADIGM

The final major distinction are the chosen programming paradigms that a language chooses to support. There are vast amounts of programming paradigms, too many to cover in this work, each with their own method of structuring programs. However, the two most commonly supported paradigms in programming languages are Object Oriented and Functional. Due to their popularity, the toy programming language has plans to support both these paradigms.

A. Object Oriented Programming

Object Oriented programming consists of objects that store data in fields and the ability to interact with other objects in a program through methods. Many programming languages support object oriented programming. These include C++, Java, Kotlin, Swift, Scala, and JavaScript. However, many of these languages are multi-paradigm languages, meaning they support more than one programming paradigm.

B. Functional Programming

Another one of the most commonly supported programming paradigms is functional programming. Functional programming consists of data and functions. In functional programming, functions are considered first class citizens and can be

passed around the same way data can be passed. This allows for more complex abstractions giving by other paradigms to be modeled as functions. Languages such as Haskell, Kotlin, Scala, C++, Javascript, and Python give extensive support for this programming paradigm.

VI. CONCLUSIONS

There are many factors that give a programming language its identity. We have only discussed the three most important characteristics that define a programming language. This work is only but scratching the surface of the many design choices that can subtly change the identity of a programming language.

REFERENCES

- [1] R. Nystrom. *Crafting interpreters*. Genever Benning, 2021.
- [2] C. Lattner and V. Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation”. In: San Jose, CA, USA, Mar. 2004, pp. 75–88.
- [3] Z. Gao, C. Bird, and E. T. Barr. “To Type or Not to Type: Quantifying Detectable Bugs in JavaScript”. In: *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. May 2017, pp. 758–769. DOI: 10.1109/ICSE.2017.75.
- [4] G. van Rossum, J. Lehtosalo, and L. Langa. *PEP 484 – type hints*. 2014. URL: <https://www.python.org/dev/peps/pep-0484/>.
- [5] M. P. Kuwabara. *RBS collection was released!* Sept. 2021. URL: <https://dev.to/pocke/rbs-collection-was-released-4nmm>.
- [6] *Placeholder type specifiers (since C++11)*. URL: <https://en.cppreference.com/w/cpp/language/auto>.
- [7] *Type inference*. URL: <https://docs.oracle.com/javase/tutorial/java/generics/genTypeInference.html>.
- [8] M. Akhin and M. Belyaev. *Kotlin Language Specification*. URL: <https://kotlinlang.org/spec/introduction.html>.

Two Modern Solutions to the Minimal-Time Control of Inverted Pendulum Problem

Xiahua Liu, *MS, Aero. Eng.*

Abstract—The minimum-time control of a swing-up inverted pendulum is a very classical problem in the optimal control theory. This paper first shows the difficulty of solving this problem using pure optimal control theory (with Pontryagin’s maximum principle). Then, 2 modern approaches are brought up. The first is direct optimization using Particle Swarm Optimization; The second is the latest reinforcement learning method, i.e. A policy-based Actor-Critic algorithm, to solve the problem. In the end, we evaluate the advantages and disadvantages of each method.

I. INTRODUCTION

The inverted pendulum problem is one of the most classic control problem in the history. Even today, it is still widely used in the control field to act as a benchmark problem for different algorithms. It can be also considered as a simplified humanoid robot walking model [1].

However, due to its inherent non-linearity in the mathematical formulation, classical control theory solutions like PID, feedback linearization, etc. which are based on the Laplace transform and the classical control theory often failed. Although, some classical control method can be considered that had roughly solved the stability control problem [2], they were not precised solutions.

The minimum-time control of the swing-up inverted pendulum problem forces finding the shortest time to swing the pole from its lowest position to highest instead of stabling the pendulum. In reality it often takes more than one sways to achieve this task, we can see this brings extra trouble to the problem because each end of the sway reaches a singularity co-state in the Hamiltonian equations.

A. Related Research

There was an analytical solution to this problem with 2-step sways by Mason et al. [3]. The solving process for a 2-step switching is overwhelmingly complicated, besides the same approach cannot be used when there are more than 2 switching points, because it assumed that there were only two switching points at the beginning. However, this research yielded an undoubtedly exact solution. In this paper, it serves as the reference for evaluating all other methods.

Also, some researcher tried solving this problem using different approaches rather than the optimal theory itself. For example, Wang et al. [4] showed their solution using an impulsive input. The paper introduce a new idea by considering the solution as a series of impulses with optimized time widths.

Nowadays as the machine learning is becoming a hot topic, many modern solutions are implemented using machine learning algorithms, such as the solution from Lillicrap et

al. [5]. This paper also includes the solution obtained from the latest reinforcement learning technique called Actor-Critic algorithm. However, we can see in the result section that it was inevitably trapped inside the local convergence points during the optimization step, the performance was not as good as the analytical solution.

B. The Improvement

In this paper, the problem will be formulated same as many previous optimal control problems, i.e. in the Hamiltonian form. However, this paper does not aim at finding a pure analytical solution for every problem, instead finding a numerical solution using modern optimization algorithms. It can be proved that the solution will converge to the exact solution eventually, and compared with modern machine learning method, it is far more precised.

II. PROBLEM FORMULATION

As can be seen in Fig. 1, a heavy steel ball is linked with the cart by a rigid stick on the vertical plane. The joint at the cart end can rotate freely without friction. The length of the stick is ℓ , the ball the cart have mass of m and M respectively. The stick’s mass is considered to be zero. The cart can only move in forward and backward directions and there is no friction between the rail and the cart. The system has an input of force, however the maximal force is F_{max} , the question is finding the shortest time control to swing the pendulum from its initial downward position up to the straight upward position. i.e. Find the optimal control of u to move $\theta: 180^\circ \rightarrow 0^\circ$ on the time interval $[0, T]$, which minimizes:

$$\mathcal{J} = \int_0^T dt \quad (1)$$

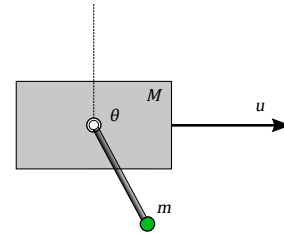


Fig. 1. Swing-Up Inverted Pendulum Control

First we need to find the equation of motion in the problem, the Lagrangian in this problem can be written as:

$$\begin{aligned}
\mathcal{L} &= T - V, \\
T &= \frac{1}{2}Mv_1^2 + \frac{1}{2}mv_2^2, \\
V &= mgl \cos \theta,
\end{aligned} \tag{2}$$

Eliminate v_1, v_2 , we have:

$$\mathcal{L} = \frac{1}{2}(M+m)\dot{x}^2 - m\ell\dot{x}\dot{\theta}\cos\theta + \frac{1}{2}m\ell^2\dot{\theta}^2 - mgl\cos\theta \tag{3}$$

According to Lagrangian mechanics [6], the equations of motion are:

$$\begin{aligned}
\frac{d}{dt}\frac{\partial \mathcal{L}}{\partial \dot{x}} - \frac{\partial \mathcal{L}}{\partial x} &= u, \\
\frac{d}{dt}\frac{\partial \mathcal{L}}{\partial \dot{\theta}} - \frac{\partial \mathcal{L}}{\partial \theta} &= 0,
\end{aligned} \tag{4}$$

Substituting $\mathcal{L}$ in Equation 4 leads to the following equation set:

$$\begin{aligned}
(M+m)\ddot{x} - m\ell\ddot{\theta}\cos\theta + m\ell\dot{\theta}^2\sin\theta &= u, \\
\ell\ddot{\theta} - g\sin\theta &= \ddot{x}\cos\theta,
\end{aligned} \tag{5}$$

Formulate the state vector as $x = [x_1 \ x_2 \ x_3 \ x_4] = [x \ \dot{x} \ \theta \ \dot{\theta}]$ and rewrite the previous dynamics system in its matrix form:

$$\begin{bmatrix} M+m & -m\ell\cos\theta \\ -\cos\theta & \ell \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} u - m\ell\dot{\theta}^2\sin\theta \\ g\sin\theta \end{bmatrix}, \tag{6}$$

This is a coupled system, using Cramer's rule to calculate the result:

$$\begin{aligned}
D &= (M+m)\ell - m\ell\cos^2\theta, \\
\begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} &= \frac{1}{D} \begin{bmatrix} u\ell - m\ell^2\dot{\theta}^2\sin\theta + gml\sin\theta\cos\theta \\ (M+m)g\sin\theta + u\cos\theta - m\ell\dot{\theta}^2\sin\theta\cos\theta \end{bmatrix},
\end{aligned} \tag{7}$$

In the Equation 7, only $\theta \ \dot{\theta}$ are needed, here I re-defined the state vector as $x = [\theta \ \dot{\theta}]$. Aside from dynamic equations, we also have multiple boundary constraints, the pendulum starts at its lowest position and ends at the highest position, at both beginning and ending, the $\dot{\theta}$ should be zero. Here all the conditions are combined.

With the same fashion showed in the old research, the system is simplified at first, let's assume the cart has no mass and our input is the accelerate of the cart, and $mg = 1$ for further calculation convenience. This problem can be written as the following form:

$$\arg \min_u \mathcal{J} = \int_0^T dt \tag{8}$$

s.t.

$$\begin{aligned}
\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} &= \begin{bmatrix} x_2 \\ \sin x_1 + u \cos x_1 \end{bmatrix}, \\
x(0) &= [\pi \ 0]^\top, \\
|u(t)| &\leq 1, \\
x_1(T) &= [0]^\top
\end{aligned} \tag{9}$$

A. Hamiltonian

Introduce Lagrange multipliers λ in the performance index.

$$\begin{aligned}
\mathcal{J}' &= \nu^\top \psi(x(T), T) + \int_0^T \mathcal{H}(x, u, \lambda, t) - \lambda^\top \dot{x} dt, \\
H &= 1 + \lambda_1 x_2 + \lambda_2 (\sin x_1 - u \cos x_1),
\end{aligned} \tag{10}$$

With Pontryagin's Minimum Principle, the optimal control is obtained when:

$$\lambda_*^\top f(x^*, u^*, t) < \lambda_*^\top f(x^*, u, t) \tag{11}$$

It can be transformed into the following form:

$$\lambda_2^* u^* \cos x_1^* \leq \lambda_2^* u \cos x_1^* \tag{12}$$

From here, it can be seen that the optimal control of the problem has a bang-bang structure:

$$u(t) = \begin{cases} -u_0, & \lambda_2^* \cos x_1^* > 0 \\ \text{any}, & \lambda_2^* \cos x_1^* = 0, \\ u_0, & \lambda_2^* \cos x_1^* < 0, \end{cases} \tag{13}$$

The co-state equations are:

$$\begin{aligned}
\dot{\lambda}_1 &= -\lambda_2 (\cos x_1 + u \sin x_1), \\
\dot{\lambda}_2 &= -\lambda_1,
\end{aligned} \tag{14}$$

From this it can be seen that switching occurs only if $\lambda_2 = 0$ or $x_1 = k\pi + \pi/2, K \in \mathbb{Z}$.

III. PHASE PLANE ANALYSIS

The phase plane plot of this problem is very different from a linear control problem. From Fig. 2, when the input u is fixed, the trajectory of any initial state can be either:

- Looping in a circle. This means the pendulum swings forward and backward in a certain period.
- Moving towards infinite θ . This means the pendulum rotates clockwise or anti-clockwise in full circles.

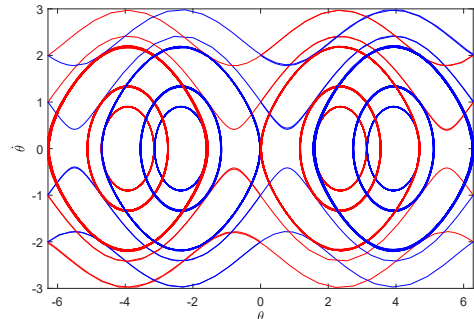


Fig. 2. Phase Plane Plot. Blue: $u = +1$, Red: $u = -1$

A. Switching Curve

A 2-step switching curve of this problem had been found in the previous research [3] and is shown below. However the result is rather complicated even for a 2-switching problem, and not applicable for more than 2-switching cases. However when the mass values are changed, the switching can happen 5 or 6 times. In other word, the whole analytical switching curve was never found by anyone, except in simple cases.

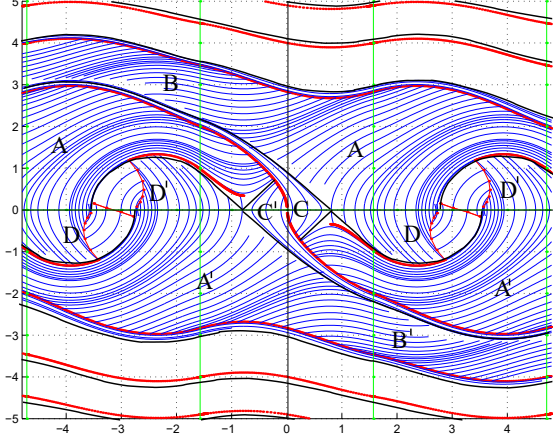


Fig. 3. The complete time optimal phase plane plot [3].

In order to evade the difficulty of this issue, let's see this problem from another viewpoint. All we need to know is the initial $\lambda(0)$, but it cannot be solved easily because the co-state functions contains the control vector. Common shooting method does not work in this problem.

Instead, here used the Particle Swarm Optimization tool [7] to finding $\lambda_1(0)$ and $\lambda_2(0)$ as a nonlinear optimization problem.

The benefit is we do not make any assumption about the switching times like previous research did, although the result fits the previous research, i.e. 2 switching. PSO has a good converge efficiency, it only cost a few minutes to get the final result in this problem. The result is shown in Fig. 4.

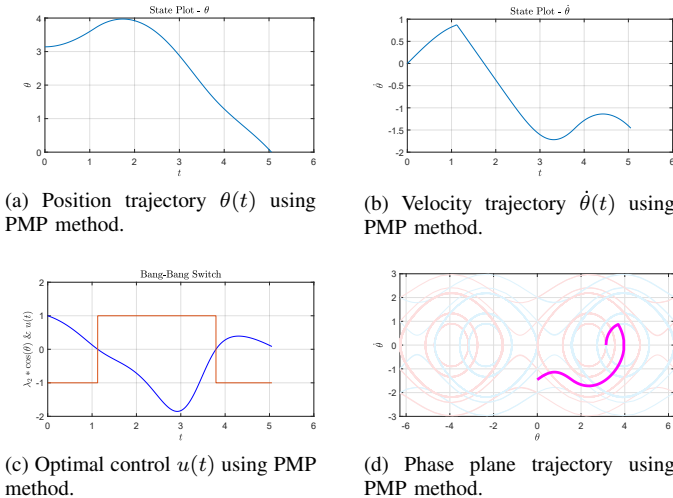


Fig. 4. Results Using PMP-PSO Method

IV. REINFORCEMENT LEARNING METHOD

It has been theoretically proved that the switching curve exists in this problem, although the switching curve is not a continuous or linear function, but a combination of few different curves. According to this fact, there exists a corresponding optimal control vector for any given state pair in the system $[\theta, \dot{\theta}]$. The optimal control vector can be written as a control policy:

$$\mu(x) \in \{-1, 1\} \quad x = \begin{bmatrix} \lambda \\ \lambda \end{bmatrix}, \quad (15)$$

$\mu(x)$ is a deterministic policy function, given a state in the phase plot it output the action, it works the same as a closed-loop form control function $u = \mu(x)$.

A. Dynamic Modeling

In order to apply reinforcement learning technique in this problem, first, an interactive environment has to be implemented. The reinforcement learning agent does not know anything about the model itself at first, meaning it is a model-free learning method.

The time step is set to be 0.01 seconds. At each step, the control vector is not changed, and the output of the system is calculated by using first order derivative in Equ. 16. Even though first-order integral has a large accumulated error, it is fast. Due to the fact that the humble computational power the author's PC has, this paper only shows the result obtained using first order derivatives.

$$\begin{aligned} \theta[n+1] &= \theta[n] + dT\dot{\theta}[n], \\ \dot{\theta}[n+1] &= \dot{\theta}[n] + dT(\sin \theta[n] + u[n] \cos \theta[n]), \end{aligned} \quad (16)$$

B. Actor-Critic Algorithm

By doing policy gradient, we find the gradient of objective function [5], in our application, the time cost function our the objective, with respect to the parameters in $\pi^\theta(x)$ [8]. In the end, we have Equ. 18.

$$J(\theta) = \sum_{s \in \mathcal{S}} d^\pi(s) V^\pi(s) = \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s, a), \quad (17)$$

$$\begin{aligned} \nabla_\theta J(\theta) &\propto \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} Q^\pi(s, a) \nabla_\theta \pi_\theta(a|s) \\ &= \sum_{s \in \mathcal{S}} d^\pi(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^\pi(s, a) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\ &= \mathbb{E}_\pi [Q^\pi(s, a) \nabla_\theta \ln \pi_\theta(a|s)] \end{aligned} \quad (18)$$

In the Actor-Critic algorithm, the expected step reward alongside the given policy $Q^\pi(s, a)$ is estimated by the Critic agent, which is another neural network, showed in Fig. 6. The result is shown in Fig. 5 and Fig. 7.

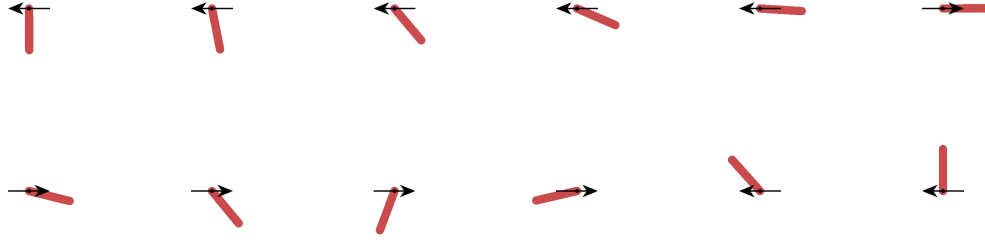


Fig. 5. Simulation of the result from RL. We can see there are 2 switching points.

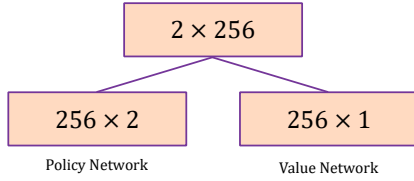
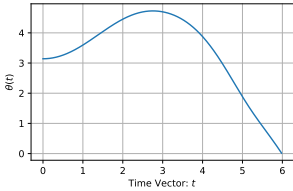
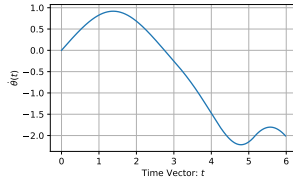


Fig. 6. Actor-Critic Neural Network

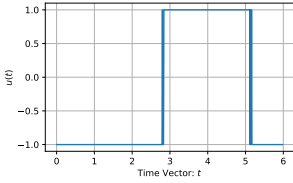
Fig. 7. Results using RL method.



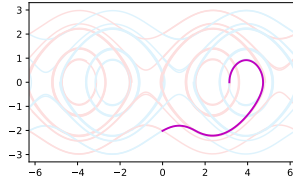
(a) Position trajectory $\theta(t)$ using RL method.



(b) Velocity trajectory $\dot{\theta}(t)$ using RL method.



(c) Optimal control $u(t)$ using RL method.



(d) Phase plane trajectory using RL method.

V. CONCLUSION

In the PMP-PSO method, we first follow the common ways to formulate Hamiltonian and solve for the co-state vector with Particle Swarm Optimization algorithm. Then a novel method using machine learning is also applied to solve this problem. Unlike the PMP-PSO solution, the designer of the robot does not need to know the dynamic characteristic of the system and the agent can learn from the objective function by itself. Because it does not rely on the parameter of the model and adopts a probabilistic control plan, it has high error tolerance. However the agent performs bad near the switching points on the phase plane. The overall performance comparison can be found in Table I.

TABLE I
PMP METHOD AND REINFORCEMENT LEARNING METHOD COMPARISON.

Method	Training Speed	Accuracy	Robustness
PMP-PSO	5 minutes	High	Low
RL	46 minutes	Low	High

REFERENCES

- [1] S. Kajita et al. "The 3D linear inverted pendulum mode: a simple modeling for a biped walking pattern generation". In: *Proceedings 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems. Expanding the Societal Role of Robotics in the the Next Millennium (Cat. No.01CH37180)*. Vol. 1. 2001, 239–246 vol.1. DOI: 10.1109/IROS.2001.973365.
- [2] E. S. Varghese, A. K. Vincent, and V. Bagyaveereswaran. "Optimal control of inverted pendulum system using PID controller, LQR and MPC". In: *IOP Conference Series: Materials Science and Engineering*. Vol. 263. 5. IOP Publishing, 2017, p. 052007.
- [3] P. Mason, M. Broucke, and B. Piccoli. "Time Optimal Swing-Up of the Planar Pendulum". In: *IEEE Transactions on Automatic Control* 53.8 (2008), pp. 1876–1886.
- [4] Z. Wang, Y. Chen, and N. Fang. "Minimum-time swing-up of a rotary inverted pendulum by iterative impulsive control". In: *Proceedings of the 2004 American Control Conference*. Vol. 2. 2004, 1335–1340 vol.2.
- [5] T. P. Lillicrap et al. "Continuous control with deep reinforcement learning". In: *arXiv preprint arXiv:1509.02971* (2015).
- [6] H. Goldstein, C. Poole, and J. Safko. *Classical mechanics*. 2002.
- [7] J. Kennedy and R. Eberhart. "Particle Swarm Optimization". In: *Proceedings of ICNN'95-international conference on neural networks*. Vol. 4. IEEE. 1995, pp. 1942–1948.
- [8] V. R. Konda and J. N. Tsitsiklis. "Actor-critic algorithms". In: *Advances in neural information processing systems*. Citeseer. 2000, pp. 1008–1014.

Design of a Learning Module for Remote Areas

Natalia Lui, *BS, Elec. Comp. Eng.*

Abstract—The following paper will be detailing the engineering design process of a learning module for remote areas. The efforts of this project used the engineering design method and standard engineering project management tools learned by a diverse group of first-year engineering majors and fourth and fifth year students in Ghana and Ethiopia. The deployment of this low-cost device is for remote areas with low literacy rates with the result of improving the state of living for the chosen population.

I. INTRODUCTION

DUE to the COVID-19 pandemic, numerous international educational programs were halted and their funding went unused. With the emerging success of Zoom in the academic sphere, Cedrick Kwuimy, an assistant professor-educator in UC's department of engineering education, and a member of UC international's sub-Saharan African group created a program to give first-year engineering students similar experiences of a work or study abroad program. These projects were submitted to the virtual international collaborative experiential program (VICEP) and received praise.

With the assistance of Dan String, a teaching assistant at the College of Engineering and Applied Science, three groups of first-year engineering students teamed with students in Ghana and Ethiopia to create modules based on Raspberry Pi 3s and electrical components for remote areas.

II. BACKGROUND

The cycle of poverty in impoverished regions such as Haiti comes with its location and lack of education. Due to the numerous natural disasters that plague Haiti's land, infrastructure and thereby, education, is difficult to upkeep. This cycle of repairing, rebuilding, and destruction gives no time for citizens to learn and thus, the cycle of poverty persists. The implementation of this portable device allows its citizens to continue learning without a traditional classroom. By increasing the literacy rate in Haiti, there is hope that the cycle breaks and the area will be encouraged to grow.

III. LEARNING MODULE

A. Participants

Student Participants

- Stephen Awuku, *Engineering Physics*
- Catie Brutvan, *Aerospace Engineering*
- Natalia Lui, *Electrical and Computer Engineering*

Supervisors

- Cedrick Kwuimy, *Assistant Professor-Educator*
- Dan String, *Teaching Assistant*
- Kwadwo A. Dompheh, *Professor at the University of Cape Coast in Ghana*

B. Objectives of the Module

The module was built to teach children how to read, write, and speak in various languages, the test language being English. For us, success looked like a working module that turned on and displayed numbers and letters. There were three objectives during the development of this module:

- Build a hardware device for learning
- Write code that allow sensors to interact with the device
- Write code for an LED matrix to display words, numbers, and letters
- Write code to play an audio recording of the corresponding words, numbers, and letters displayed on the device

C. Accessibility

The flexibility of this learning module is in its accessibility. It can be used by any age, can teach any language, and, in future implementation, simply require sunlight to power the device through the use of solar technology.

D. Project Management

In terms of project management, our goal was to make the device easy-to-use and relatively cheap for remote areas around the world. We used Slack to manage communication across all team members and to share documentation. Zoom was the primary platform for meetings, and a Gantt chart was used to determine timeline and workload.

Our in-person meetings were with Cedrick Kwuimy and Dan String on UC's campus in the Rhodes building. We used that time to bring in our module and work on the hardware components alongside the software components. Thanks to the funding this program received from UC International, we had two Raspberry Pi 3s, one for hardware and one for testing dummy code, to speed up the work. Because of this financial assistance, the learning module was able to be completed during the spring semester of 2021 with minor hassles.

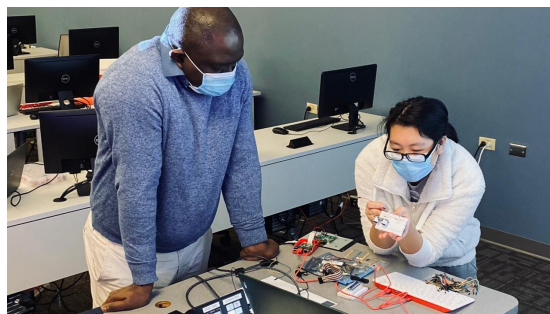


Fig. 1. Cedrick Kwuimy with Natalia Lui holding a Raspberry Pi 3. Photo by Dan String.

E. Challenges

Being first-year students, there was a significant learning curve with the Raspberry Pi 3 hardware, LED matrix hardware and software, publishing code to GitHub, and gaining familiarity with Python. Throughout the spring semester, the skills and technologies learned from the course, Foundations of Engineering Design Thinking II (ENED1120), were immediately implemented into the learning module. The initial issues with executing these skills was the gap in experience and the frustration of not fully understanding certain concepts.

As with most projects, there are bound to be obstacles. The largest complication faced was the delivery of certain components. The learning module required an LED matrix that took two weeks to deliver from Amazon. Once delivered, its electrical components short-circuited when plugged into a wall outlet and tested with the dummy code. Ordering a second LED matrix set the project back by about two more weeks.

F. International Challenges

Our counterpart in Ghana, Stephen Awuku, was working on an Arduino version of the learning module. However, due to the restrictions on air travel, he had difficulties with getting all the supplies and finishing the project on time. Due to the lack of resources, Stephen wired all two hundred LEDs.

There were also issues with the four hour time zone difference between Cincinnati, Ohio and Cape Town, Ghana. It made online meetings very difficult to execute, even with the comfort of Zoom. Because of this, the project timeline was often thrown off and we had to communicate heavily to get back on track. The addition of classes and labs made this much more difficult. Throughout this experience we learned to take mistakes and complications in stride and to keep progressing.

What we didn't realize in the initial stages of testing was that most of the issues were hardware rather than software. With our inexperience, most of the blame was shifted onto the software. However, multiple tests with dummy code and responsiveness of specific devices showed that there were inconsistencies with the power, wiring, and soldering of the Raspberry Pi 3 that was redone over and over again.

H. Future Improvements

Due to a lack of time, the Raspberry Pi team was unable to implement the audio recording that played with each letter and word that came across the LED matrix. Furthermore, there were complications when implementing the touch modules with code on the LED matrix. Given more time, and a mentor, these features could be added.

During the initial discussion of the learning module, we wanted to make it portable, compact, and solar powered. Again, given time and resources we could replace the wall outlet power source with a battery and solar panel. This would allow the device to be a much more compact shape and, in turn, portable. The current portable learning module consists of three separate rectangular components (LED matrix, Raspberry Pi 3, and breadboard) connected by wiring and soldering. With these future improvements, the learning module could assume the unique, robust shape of a triangular prism.

I. Results

Overall, the initial purpose of this collaboration was fulfilled and all students involved gained experience in working with students abroad while exploring and developing solutions for remote areas. The approach was unique and took advantage of the resources available during a global pandemic.

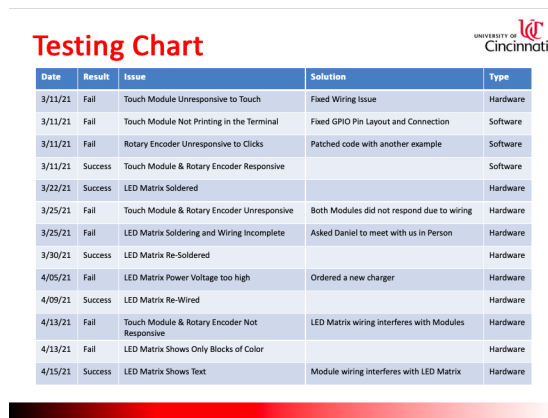
This learning module was presented at Penn State's Center for Global Engineering Engagement's Virtual Spring 2021 Conference, "Engineering Your Global Experiences: Building Global Competencies for the Future Engineering Workforce" and at the Undergraduate Research Showcase at UC.

Submissions for the Undergraduate Scholarly Showcase were made public on the University of Cincinnati website¹.

IV. CONCLUSION

The impact of this experience reaches past the physical barriers of travel, communication, and collaboration to the first-years that participated. Researching remote areas and learning about literacy rates to create a device that can improve the quality of living for those same areas allowed a greater understanding of engineering design processes, developing products, and providing for those in need. It is an invaluable experience that can only happen once in a global pandemic.

¹<https://www.uc.edu/campus-life/careereducation/get-experience/research/-showcase/2021-presentations/category-b.html>.



Date	Result	Issue	Solution	Type
3/11/21	Fail	Touch Module Unresponsive to Touch	Fixed Wiring Issue	Hardware
3/11/21	Fail	Touch Module Not Printing in the Terminal	Fixed GPIO Pin Layout and Connection	Software
3/11/21	Fail	Rotary Encoder Unresponsive to Clicks	Patched code with another example	Software
3/11/21	Success	Touch Module & Rotary Encoder Responsive		Software
3/22/21	Success	LED Matrix Soldered		Hardware
3/25/21	Fail	Touch Module & Rotary Encoder Unresponsive	Both Modules did not respond due to wiring	Hardware
3/25/21	Fail	LED Matrix Soldering and Wiring Incomplete	Asked Daniel to meet with us in Person	Hardware
3/30/21	Success	LED Matrix Re-Soldered		Hardware
4/05/21	Fail	LED Matrix Power Voltage too high	Ordered a new charger	Hardware
4/09/21	Success	LED Matrix Re-Wired		Hardware
4/13/21	Fail	Touch Module & Rotary Encoder Not Responsive	LED Matrix wiring interferes with Modules	Hardware
4/13/21	Fail	LED Matrix Shows Only Blocks of Color		Hardware
4/15/21	Success	LED Matrix Shows Text	Module wiring interferes with LED Matrix	Hardware

Fig. 2. Testing Chart of Learning Module. Screenshot by Natalia Lui.

G. Prototyping and Testing

When it came to prototyping, there were both hardware and software issues. Figure 2 documents the date, result, issue, solution, and type of issue that was experienced during prototyping and testing. All of the code that was used is documented on GitHub (<https://github.com/20NataliaLui/LearningModule>).

Having the Entrepreneurial Mindset as a Computer Scientist

Jay Kothari, *BS, Astrophys., Cert, Quant. Comp.*

Abstract—During my freshman year at the University of Cincinnati, I started as a Physics major but soon realized my passion lies in quantum computing. I want to be able to contribute to a space that has the potential to make our lives easier. Entrepreneurship is the path I chose to get there.

I. INTRODUCTION

I became fascinated with computer science after discovering Quantum Computing – the development of machines that use principles of quantum physics to store data and perform computations. Even though it felt like science fiction at first, quantum computing became a realistic and obtainable career choice I fell in love with. The technology is so new that only a handful of companies have their own quantum computers, which are unfortunately full of errors and noise. I became excited by the idea of providing solutions for a new and emerging field unknown to most. Rather than working for someone else, I want to use inspiration, passion, and excitement to start a successful quantum computing startup. However, I am not naïve to the fact that I do not currently possess the quantum physics or entrepreneurial expertise necessary to start such an endeavor. Hence, I started my journey to join the entrepreneurial community at UC while self-studying quantum computing.



Fig. 1. Picture with President of UC at Lindner Award Ceremony

II. ASPECTS OF GREAT ENTREPRENEURS

After researching successful entrepreneurs like Elon Musk, Mark Zuckerberg, and Bill Gates, I realized that most of them were engineers with a deep understanding of Computer Science. They leverage their knowledge to solve problems faced by common people in a manner that brought them success.

A. Best in the World at Something

My studies are my top priority because I learn from experts. But again, I make sure that classes are not the only source of

knowledge. In the age of the internet, I try to make the most out of the resources that are available out there. Above all, I think that the main source of knowledge and information is books. Books are the culmination of someone's life work, their biggest accomplishments, their biggest mistakes – everything compiled into 200 pages. You can get insights into their whole lives in just 10 hours.

B. Create Services and/or Products that Add Value to Society

I found the quickest way to start off my journey was by creating a blog where I share my discoveries and knowledge with the world. Through my blog, I am building a legacy where I share my passion, journey, ideas, and lessons learned through life, books, and coursework. The focus of my blog is to create a space where everyone can learn, grow, and create their own legacy. My blog does not just help me add value and create an audience, but it helps me improve my communication skills, learn about web development, web design and search engine optimization – the skills that will be invaluable when I have my own company and its journey is documented at <http://www.ourlegacy.blog/>.

C. Have an Audience

Another particularly important thing that I learned through my short journey so far is – “you are the average of the 5 people you spend the most time with”. Honestly speaking, the idea of starting my own company seemed impossible. I thought it was reserved for the already rich or lucky ones. But after I got my job at the Center for Entrepreneurship, I actively started working with entrepreneurs, or are on their way to becoming one. Though I am an aspiring entrepreneur, I love learning and growing from actual entrepreneurs.

III. THE JOURNEY TO BEING AN ENTREPRENEUR

As I started spending more time with successful people, the idea of becoming an entrepreneur seemed more and more realistic. I started getting more invitations to events. It was not easy to just walk into the events as the youngest person with no business or entrepreneurship experience. But I did it anyways. As one of them put, “You have to have courage to come here man. I really like it.”

Now, the quote “Your network is your net worth” seems to be truer than ever. In fact, I was asked to write this article by someone I met at one of the networking events. Who knew that it would lead to this amazing opportunity?

Another great aspect of an entrepreneur is being able to make the most of a given opportunity, and I try to seize



Fig. 2. Picture with Dean of College of Business at Lindner Award Ceremony

these opportunities as much as possible. Take this article for example. My first thought was that I am not credible enough to write an article about “Being an Entrepreneur as a Computer Scientist”. But I also knew that an entrepreneur tries to make the most out of a given opportunity, so I said yes to writing the article and changed the prompt to – “Having the Entrepreneurial Mindset as a Computer Scientist”.

IV. CONCLUSION

The marvels of quantum computers start where classical computers stop. I look forward to improving the world with technology that can solve problems in minutes; problems that otherwise took several years to solve. If I have learned anything, it’s that I am already a startup founder. After all, entrepreneurship is a mindset.

Unity Game Development as a Computer Science Student

Samuel Scherer, *BS, Comp. Sci.*

Abstract—Game Development is a field which relies heavily on programmers, however, the UC Computer Science degree does not offer specific courses directly targeting game development. While the CS program does include some course material which aids in game development, several skills must still be learned by the students in order to create polished games.

I. INTRODUCTION

THE Game Development industry is one of the core areas in Computer Science, requiring interdisciplinary knowledge of various other fields such as Artificial Intelligence, Computer Graphics, and Optimization Algorithms [1].

The experience discussed in this paper is from a Computer Science student perspective, with emphasis on self-taught skills, ranging from computational linear algebra to Unity Engine. This paper also includes a walkthrough of a recently developed project, showing code and image examples.

II. SELF-TAUGHT SKILLS

Since this field is highly fast-paced [2], requiring an increased ability to adapt for change, I taught myself many skills before being able to dive into building polished video games, and I have condensed that learning into two abstract categories which I have listed and detailed in Sections II(A) and II(B).

A. Unity

I began teaching myself Unity at the 2018 RevolutionUC Hackathon, at which I made a short 2D platforming game called “Meme Royale”, which parodied Battle Royale games which were extremely prominent at the time. After learning the basics for that hackathon, I was able to work for a professor at UC for my first co-op, creating a Unity project from scratch.

After leaving for a different job on my following co-op, my development experience has been either in my free-time, or during Hackathons or Game Jams (game development event).

To my knowledge, there are no Computer Science classes which utilize Unity, nor are there any CS classes which teach C#, the language utilized by Unity. There are some courses in the Information Technology college which teach such subjects, however those are not class options for most CS students.

B. Acquiring Assets

After the first game I made at RevUC 2018, I realized that making games requires more than just code; art and sound/music are also extremely important for a good game.

Assets for games can be found on the internet in the public domain or with royalty-free or attribution licenses. When I can

not find an art asset which matches what I am seeking, I would design the asset myself or edit an online asset, often using the online sprite editor *Piskel*. When participating in Hackathons and Game Jams, I often try to invite teammates who are artists so that we can have high-quality jam-specific art.



Fig. 1. Snapshot of Dualikiwi – Built with 3 Non-Programmer Contributors

Learning to make custom sound is a task that I have had a harder time learning. Even though I come from a moderately music-heavy background¹, the tools used in sound design and music design were always intimidating to me. However, I recently discovered Freesound (<https://freesound.org>) for high-quality sound assets, which can be edited as needed using Audacity. An example this interface can be seen in Fig. 2.

III. HELPFUL COURSEWORK

As mentioned earlier, the Computer Science program at UC does not teach any Unity courses. Hence, the discussion in Sections III(A)–III(C) is recommended for every CS student, and was helpful in improving my game development skills.

A. Linear Algebra

According to Wikipedia [3], Linear Algebra is “the branch of mathematics concerning linear equations...and their representations in vector spaces through matrices”. When developing in Unity, the area where the game takes place, the “Scene”, is represented using 2D or 3D vector spaces and grids, and Unity C# scripts utilize Linear Algebra concepts when changing the game world. For example, I recently created a VR

¹Played Trombone for about 6 years and grew up with a musical family.

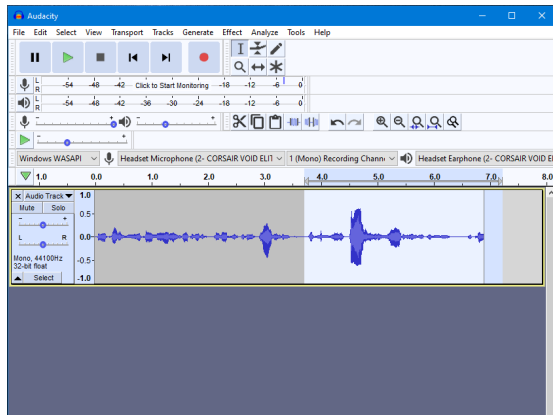


Fig. 2. Editing Audio Clips in Audacity

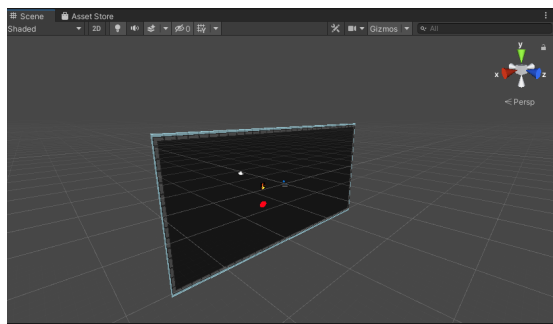


Fig. 3. Scene View in Unity – Notice x-y-z Axis Labels

Spaceship game (Fig. 4) where the user fights against enemies from the cockpit of a spaceship. When creating the enemies, I wanted to create an effect where they fly across the players' vision to allow the player an opportunity to shoot at them. I created this effect by taking the vector between the player and the enemy, performing a Cross Product on that vector and the vector representing "up" for that enemy then had the enemy turn to face in the direction of the resulting vector. I would not have been able to implement that effect if I had not paid attention to fundamental Linear Algebra concepts.

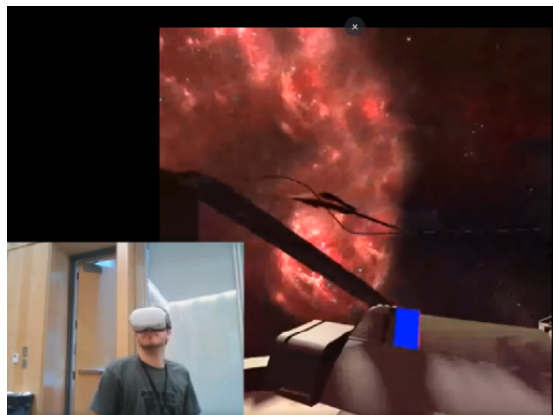


Fig. 4. Snapshot of Spaceships VR

B. Object-Oriented Programming

When creating games in Unity, it is critical to learn about Game Objects as defined by Unity. Nearly everything in the level is a Game Object. The code will be attached to those Game Objects as Components, and will be able to affect the state of those Game Objects, either once every time a frame is rendered, or once every 60 seconds.

C. Monobehaviour

In Unity, most classes inherit from a class called *Monobehaviour*. Classes inherited from Monobehaviour are able to be added as a Component and are instantiated for every existing object (on which it exists as a Component). An understanding of Object-Oriented Programming is very helpful when interacting with Game Objects and Components and is pivotal to being efficient when using Game Objects. OOP topics are taught in general CS courses, typically via C++, and are helpful when learning about C# and Game Objects.

IV. CONCLUSION

While the UC Computer Science degree does not currently offer Game Development courses, it is definitely possible for a CS student to learn to use knowledge from the provided programming courses and self-learning skills on top of that to develop games, though it may take a lot of time and effort.

REFERENCES

- [1] J. Urbain. "Introduction to Game Development". In: *Cell* 414 (2010), pp. 745–5102.
- [2] J. Blow. "Game Development: Harder Than You Think: Ten or twenty years ago it was all fun and games. Now it's blood, sweat, and code." In: *Queue* 1.10 (2004), pp. 28–37.
- [3] Wikipedia Contributors. *Linear Algebra — Wikipedia*. [Online; accessed 01-Apr-2022]. 2022. URL: http://en.wikipedia.org/w/index.php?title=linear_algebra.

Modeling Hadronization using Machine Learning

Ahmed Youssef, *Ph.D, Theo. Phys., BS, Phys.*

P. Ilten, et al., “Modeling Hadronization using Machine Learning.” *arXiv preprint arXiv:2203.04983* (2022).

WE present the first steps in the development of a new class of hadronization models utilizing machine learning techniques. We successfully implement, validate, and train a conditional sliced-Wasserstein autoencoder to replicate the Pythia generated kinematic distributions of first-hadron emissions, when the Lund string model of hadronization implemented in Pythia is restricted to the emissions of pions only. The trained models are then used to generate the full hadronization chains, with an IR cutoff energy imposed externally. The hadron multiplicities and cumulative kinematic distributions are shown to match the Pythia generated ones. We also discuss possible future generalizations of our results.

NeuroSOFM: A Neuromorphic Self-Organizing Feature Map Heterogeneously Integrating RRAM and FeFET

Siddharth Barve, *MS, Elec. Eng., BS, Elec. Eng.*

S. Barve, et al., “NeuroSOFM: A Neuromorphic Self-Organizing Feature Map Heterogeneously Integrating RRAM and FeFET.” *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* 7 no. 2 (2021): 97-105.

MANY currently available hardware implementations of the unsupervised self-organizing feature map (SOFM) algorithm utilize complementary metal–oxide–semiconductor (CMOS)-only circuits that often compromise key behaviors of the SOFM algorithm due to complexity. We propose a neuromorphic architecture harnessing the unique properties of ferroelectric field-effect transistors (FeFETs) and gated-resistive random access memory (RRAM) for in-memory computing to implement the SOFM algorithm. The FeFET-based synapse, organized in a novel circuit, is able to compute the input-weight Euclidean error in memory via the saturation drain current. The self-decaying states of the gated-RRAM allow for a self-decaying neighborhood and learning rate implementation to allow for convergence and lifelong learning. This novel architecture is able to successfully cluster benchmarks (RGB colors and MNIST handwritten digits) and real-life datasets, such as COVID-19 patient chest X-rays completely unsupervised. The architecture also demonstrates a significant amount of robustness to device variability and damaged neurons. In addition, the proposed architecture is completely parallelized and provides a power-efficient platform for implementing the SOFM algorithm.